

Intro to AI in Robotics



Vlad Shakhuro

12 February 2026

Outline

- 01 What is Embodied AI?
- 02 Evaluation: Sim and Real
- 03 Understanding the World
- 04 Planning
- 05 Acting: Manipulation
- 06 Acting: Navigation
- 07 Reward Functions
- 08 Conclusion

01



What is Embodied AI?

Embodied AI

Goal of EAI is to create intelligent agents (i.e. robots) with physical **embodiment** that can solve challenging tasks

Such agents should be able to:

- **Perceive:** see, listen, touch their environment using various sensors and extract meaningful information
- **Talk:** hold a natural dialog grounded in their environment
- **Reason:** consider and plan for the long-term consequences of their actions
- **Act:** navigate and interact with their environment to accomplish goals



General-Purpose Robots



robot dog
UniTree Go2

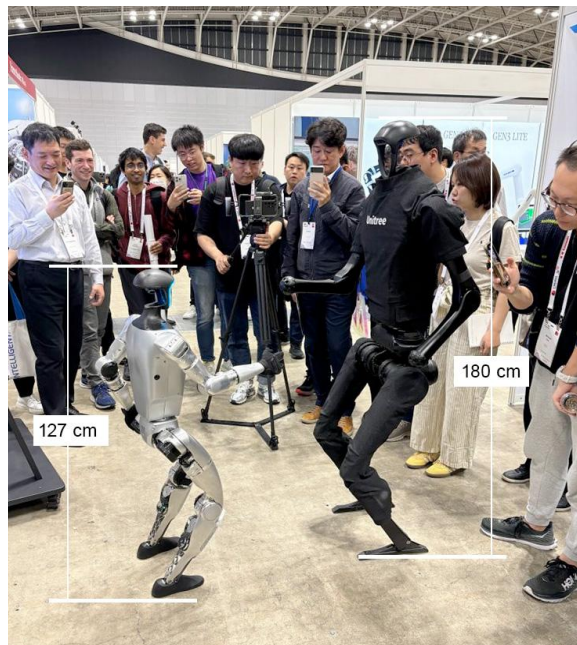


**robot dog with
hybrid locomotion**
UniTree B2-W

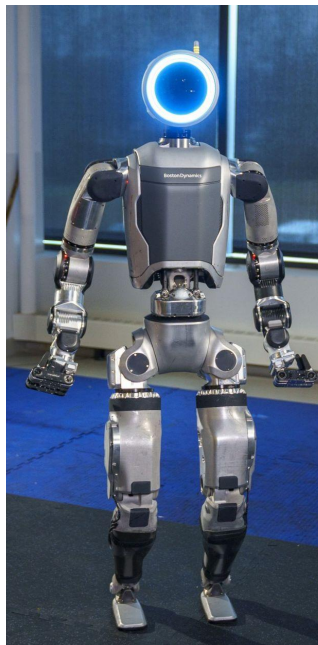


**mobile robot
with 2 arms**
TIAGo Pro

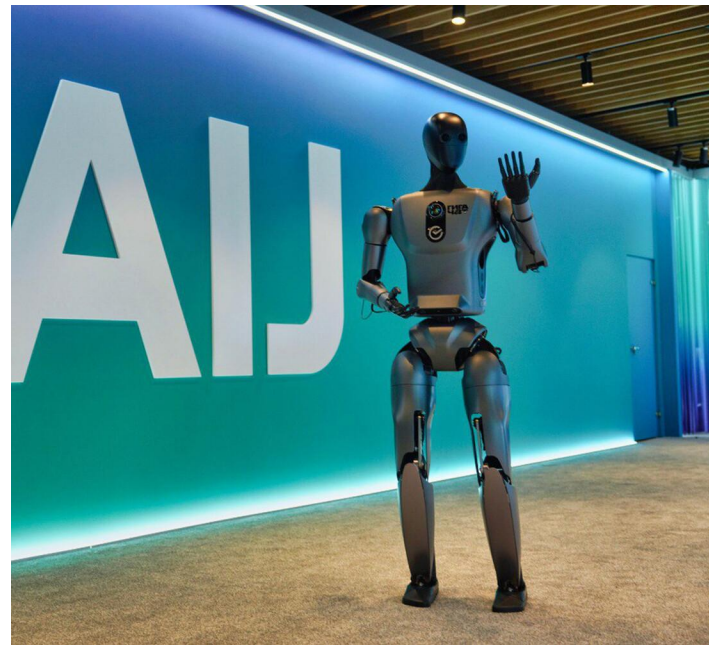
General-Purpose Robots



UniTree G1



Atlas
Boston Dynamics



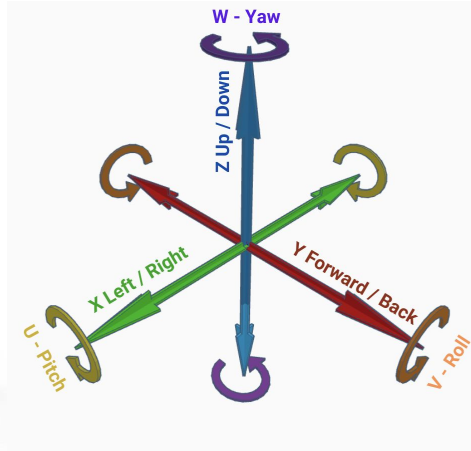
Green
Sber

General-Purpose Robots

Each embodiment is characterized by **how we can control it**



robot with gripper
7 Degrees of Freedom



autonomic vehicle
steering and acceleration

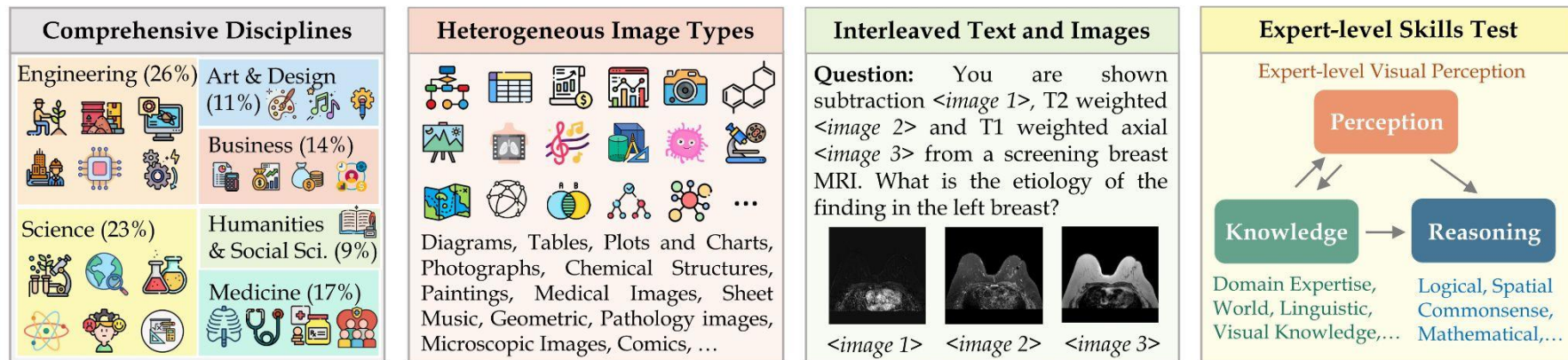
02

Evaluation: Sim and Real

CV/NLP Benchmarks

A lot of **CV/NLP benchmarks** are available for various tasks:

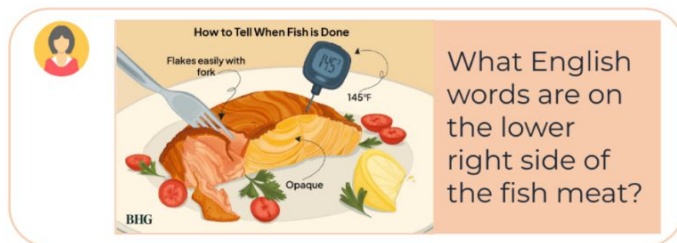
- usually very **fast evaluation**
- **reproducible**
- **cheap**
- have **big domain shift** in relation to robotics data
- may be only a **proxy metric/guidance** for choosing baseline models



Arena for Evaluating VLLMs

Rules

- Chat with two anonymous models
- Continue to chat until you identify a winner
- Vote for the better one with reason



Model A

On the lower right side of the cooked fish, the word "Opaque" is labeled.

Model B

The English word on the lower right side of the fish meat is "Opaque."



Reason

Both Model A and Model B answer correctly regarding the text.

Vote



A is Better



B is Better



Tie



Both are bad

Model A: Claude-3-Sonnet, Model B: GPT-4V

WVArena Elo Ratings



Submit

WVBench Scores



Judge
GPT-4o

Both models
are correct



Reference

Claude-3-Sonnet



Bench Data

500 sample



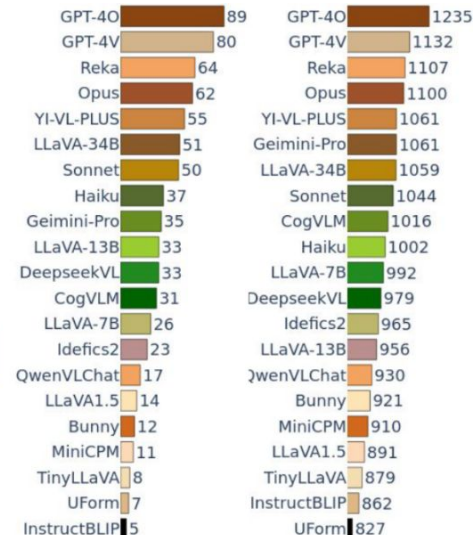
Sample Criteria

- Safety
- Diversity



Arena Data

20k+ chat 8k+ vote



WildVision
Bench

WildVision
Arena

WVArena

0.94

0.86

0.79

0.79

0.77

WVBench MMVet MMMU MMStar A1ZD

Correlation w. WVArena Leaderboard

Arena for Evaluating EAI

30.09.2025, 13:01:08 | frodobots | Policy A: pi0_fast_droid | Policy B: pi05_droid | Pref: A

Evaluator: michael.cho@frobobots.com

Instruction: fold the towel in half

Feedback: Policy A first took some time to find the target, then moved the gripper close to towel, grasped the bottom right corner of towel, lifted it up, moved it to left and release it, causing the towel to be folded although not in perfect half. Policy B first moved the gripper close to the towel, grasped and released the bottom right corner of the towel, then moved the gripper away in the end. My instruction is to fold the towel in half, when comparing the performance of the two, policy A folded the towel although not in perfect half, whereas policy B only grasped the towel but did not fold it. Therefore, policy A is obviously better than policy B.

Policy A • partial success 75.0%

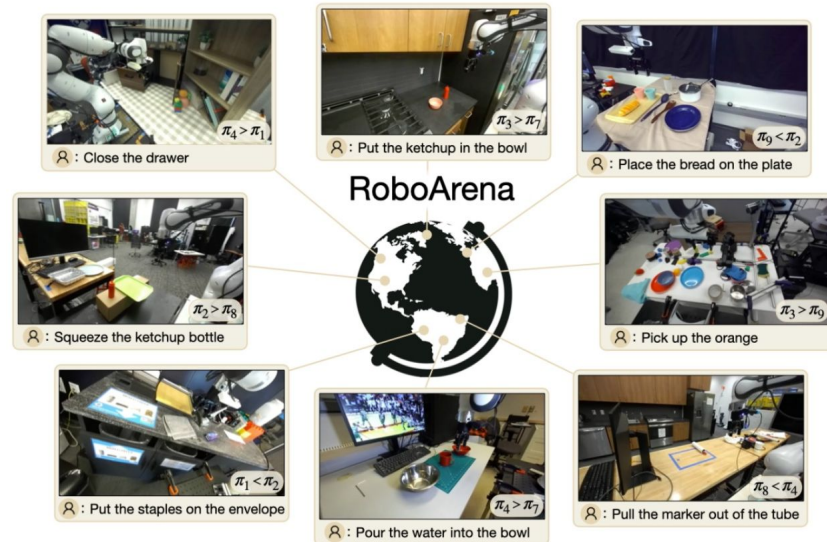


Policy B • partial success 30.0%



Distributed Network of Evaluators

(each runs A/B comparisons on whatever scene & task they choose)



Arena for Evaluating EAI

VLA
ARENA

Таблица лидеров

Симулятор

Реальный робот

Вступить
в сообщество

Делимся опытом,
разбираем поведение
моделей и обсуждаем
VLA.

Telegram

Таблица лидеров

Каждая таблица помогает быстро сравнить модели и увидеть лидеров. Рейтинг отражает общую силу по мнению сообщества (чем выше — тем лучше), Голоса показывают количество сравнений модели, а лицензия ее открытость. Мы отдельно показываем результаты в симуляции и на реальном роботе, чтобы вы видели, кто впереди в каждом сценарии.

В симуляции

У меня нет робота - сравнить модели

Ранг	Модель	ELO	Голоса	Лицензия
#1	pi-05-libero-ru-en	1598	52	Open-source
#2	pi-05-libero	1552	53	Open-source
#3	flower-libero-90	1480	26	Open-source
#4	smolvla-libero	1370	49	Open-source

На реальном роботе (LeRobot SO100 и SO101)

У меня есть робот - сравнить модели

Arena for Evaluating EAI

Build Your Own SO-101 Robot!



Meet the updated SO100, the SO-101 – Just €114 per arm!

<https://github.com/huggingface/lerobot>



Simulators: Static Scans vs CAD

Scan-based env:

- relatively fast to collect
- realistic look
- fast to render
- limited number of supported tasks



CAD-based env:

- hard to prepare
- realistic look requires a lot of effort
- may be challenging to render
- all tasks are supported

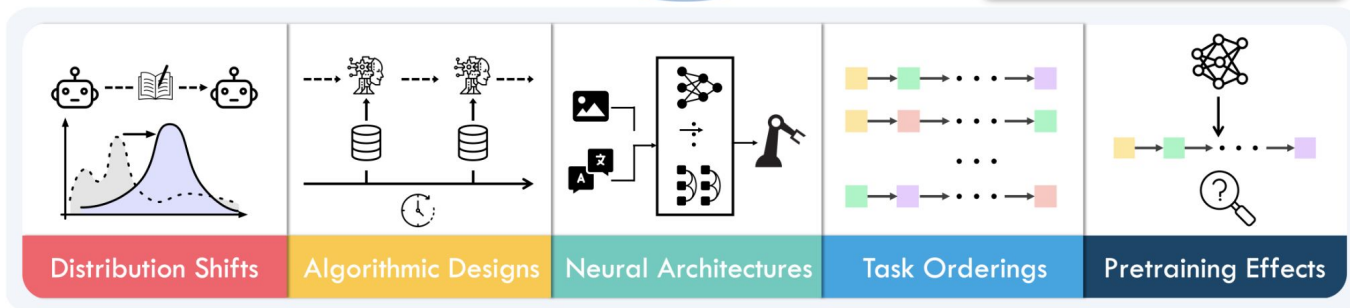
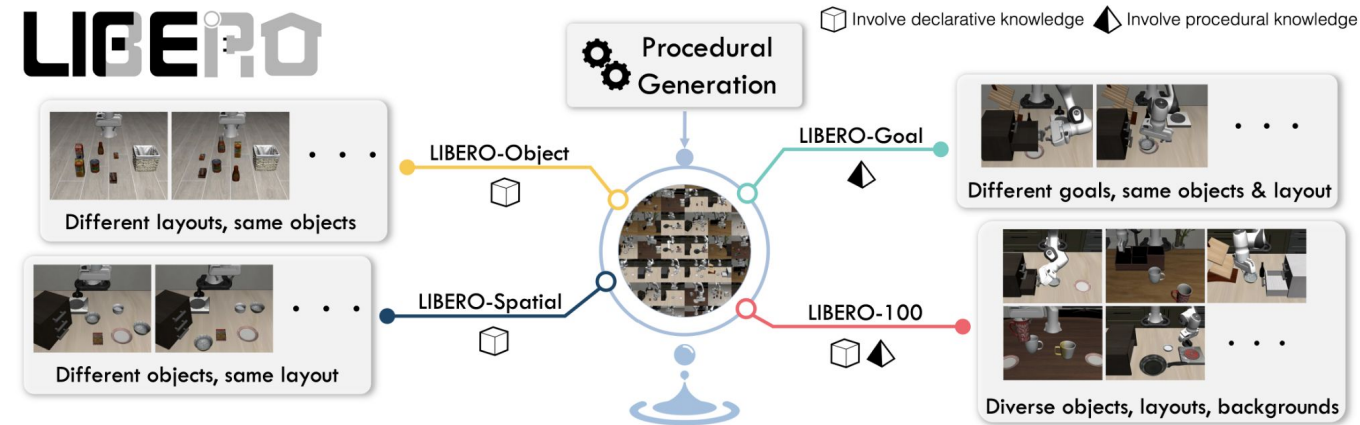


Simulators: LIBERO

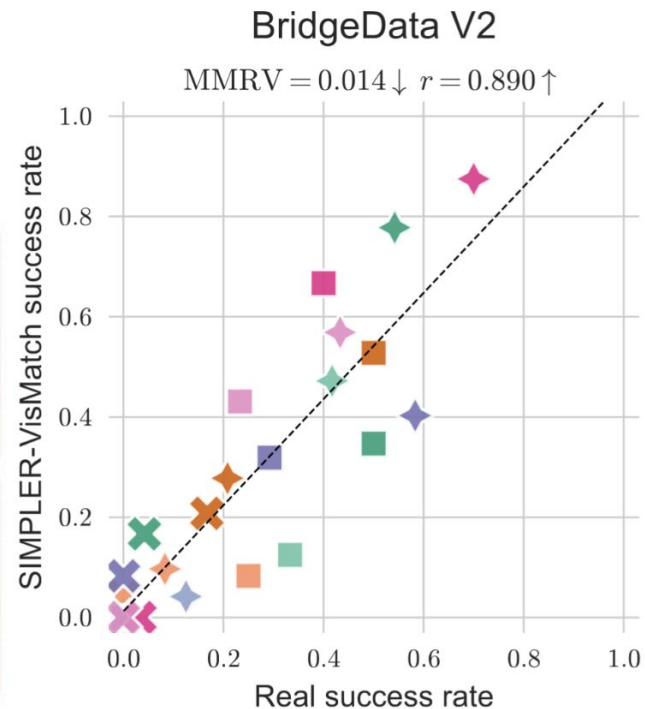
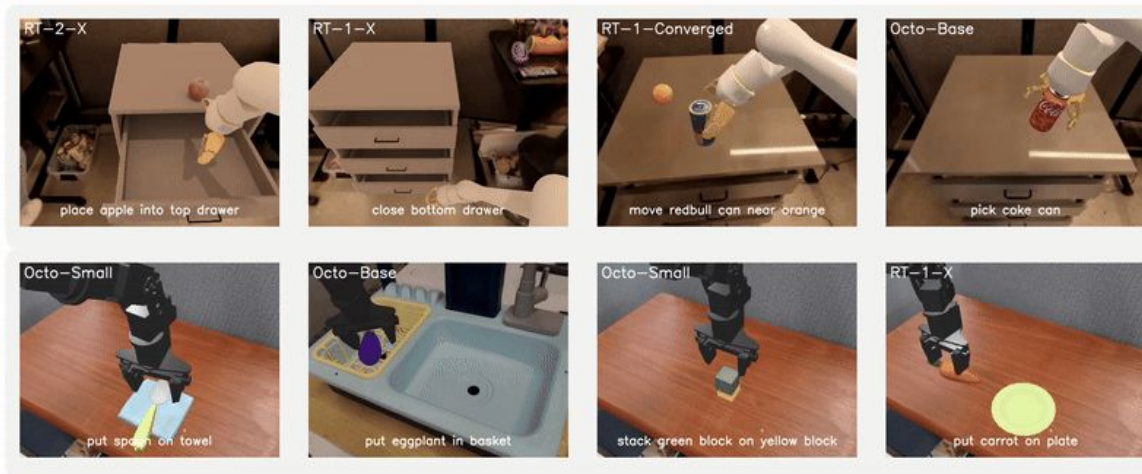
<https://libero-project.github.io>



LIBERO



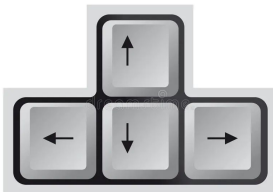
Simulators: SimplerEnv



Simulators: World Models

World model is a special case of simulators. Instead of painstakingly modeling every object and light ray, the world model **learns directly from data**

Actions:



Generate a playable world
set in a futuristic city

Simulators: World Models

- **Speed** – real-time 3D rendering plus physics is slow
- **Flexibility** – don't need CAD files of every object in every room
- **Data-driven** – world models adapt to whatever domain you train them on



world model learns directly from data

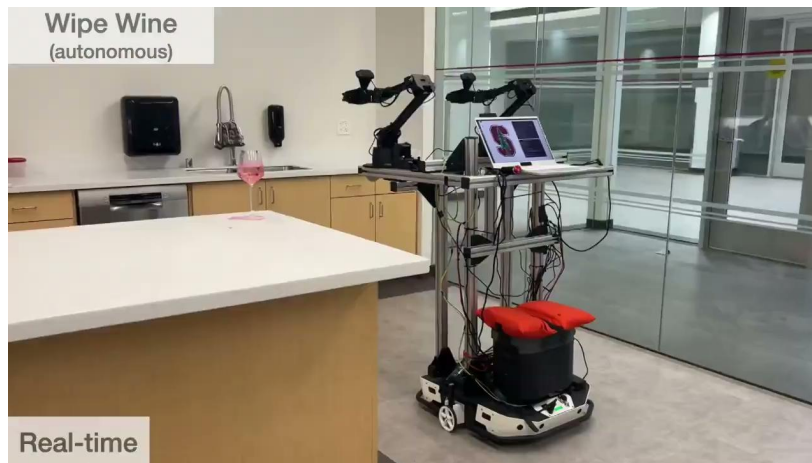
Simulators: World Models

Limited application – you need to have massive recorded interactions (gameplay video, driving logs, lab trials) with various scenarios



Real-world Evaluation

- the only evaluation that really matters
- very slow
- very expensive and technically complex
- scales badly



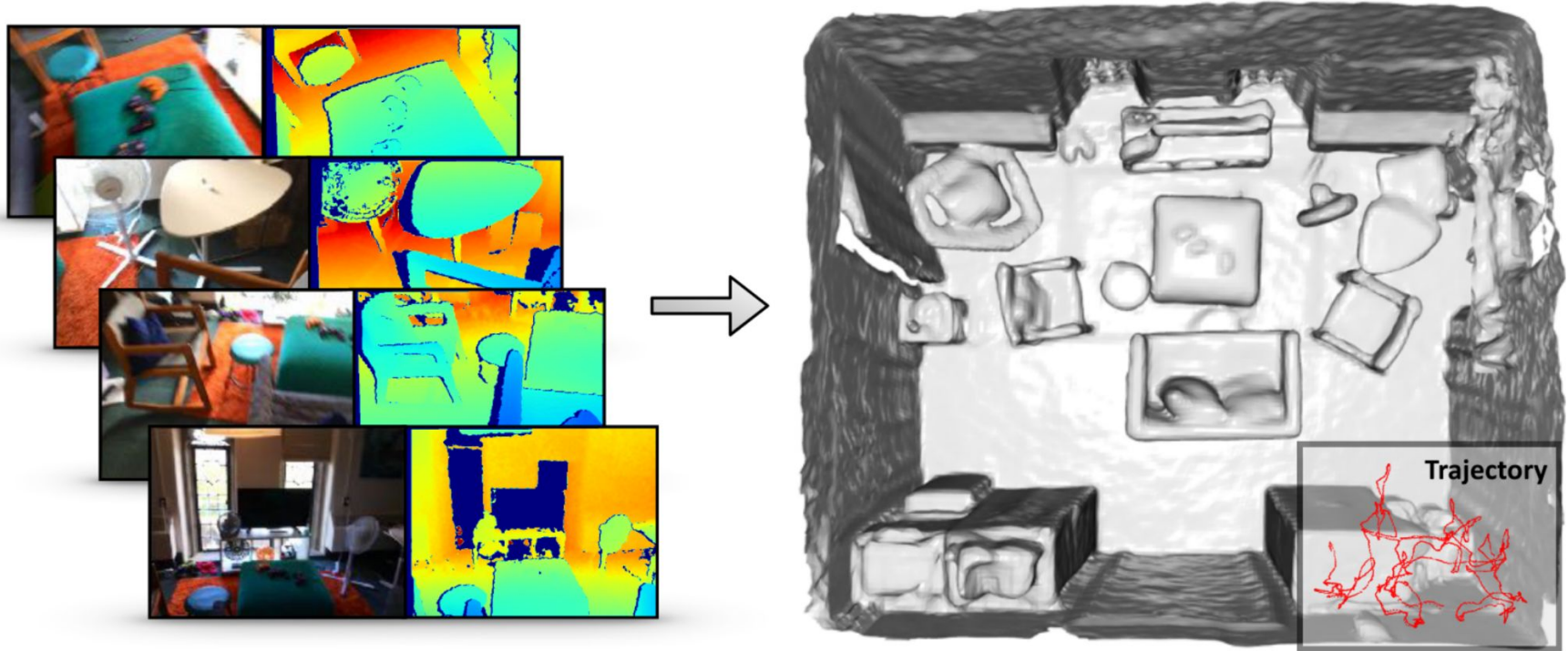
Comparison of Evaluation Methods

	Static benchmarks	Arena	End-to-end sim	End-to-end real
Relevance	Low	Medium	Medium	High
Safety	High	High	High	Low
Speed	High	Medium	Medium	Low
Cheapness	High	Medium	Medium	Low
Reproducibility	High	High	Medium	Low

03

Understanding the World

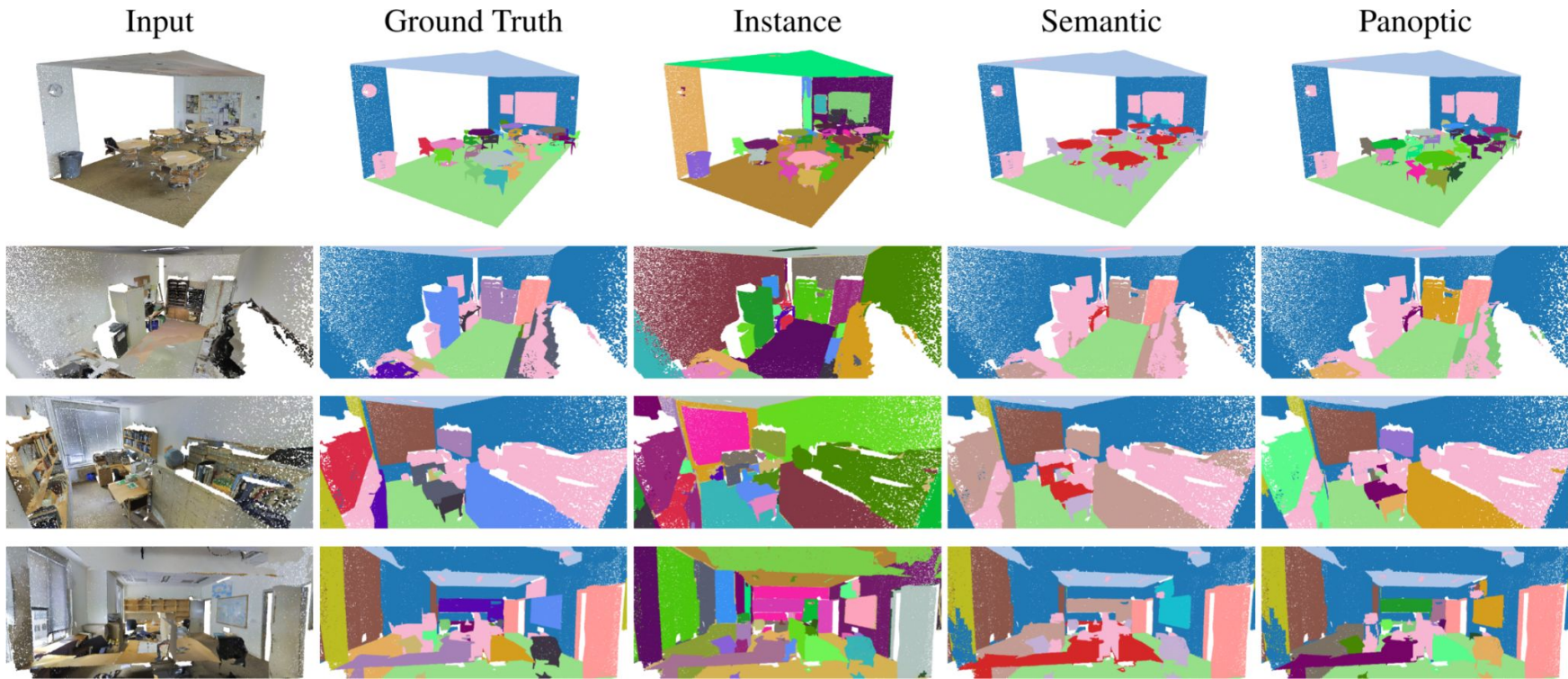
3D Reconstruction (SLAM)



- Most embodies now have RGB+Depth/LiDAR sensors
- It can be converted to the map of the area and determine robot position using SLAM

3D Segmentation

- Point clouds can be enriched with semantic information
- 3D scene representation (i.e. point cloud) can be segmented into a fixed set of classes

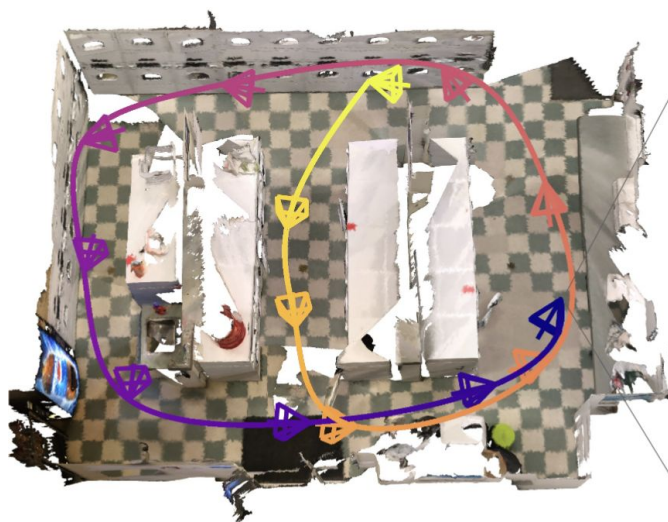


Embodied Question Answering

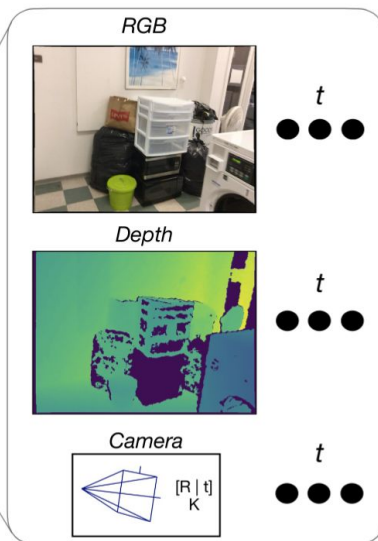
Agent has to answer questions about environment. Two scenarios are possible:

- pre-recorded video stream
- fully interactive mode: agent can freely explore environment

Evaluation of answers is done with GPT or via human evaluation



Environment Trajectory



Multimodal Observations

Question:

What is below the white plastic storage bin?

Answer:

Two microwaves.

Question:

Where did I leave my paper bag?

Answer:

Near two microwaves and a plastic drawer.

Question:

Where can I get some pop drinks?

Answer:

Buy some from the vending machine near the corner of the laundry room

Open-Vocabular Q&A

04

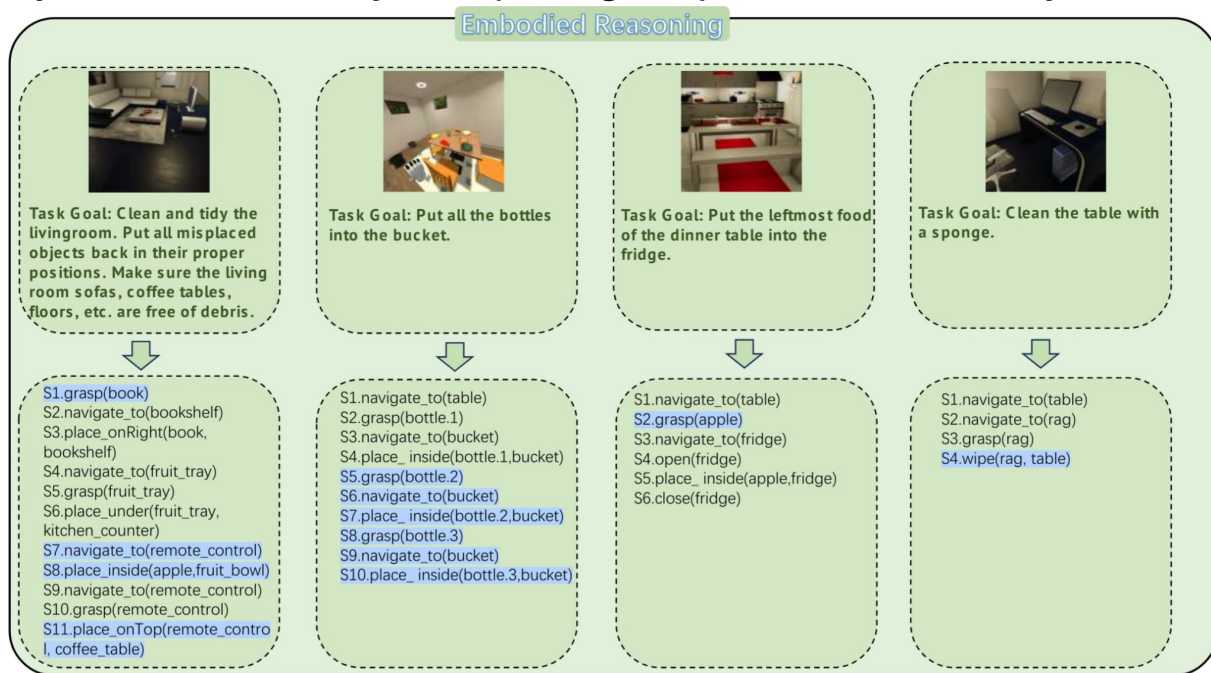
Planning

Planning

Planning is **decomposing** high-level task into sequence of sub-tasks.

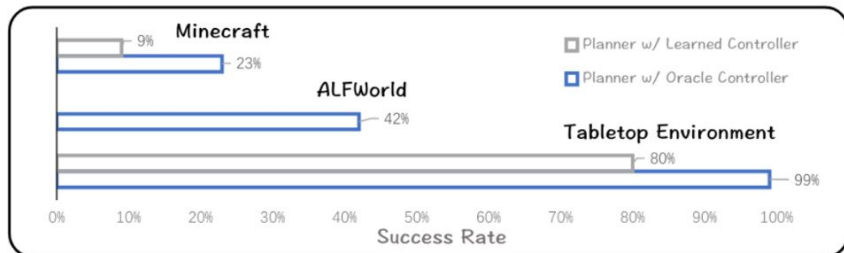
Simplest case of planning is planning with ground truth of fixed possible actions:

- obtaining ground truth is laborious
- quality may be measured by comparing sequences (accuracy or GPT score)

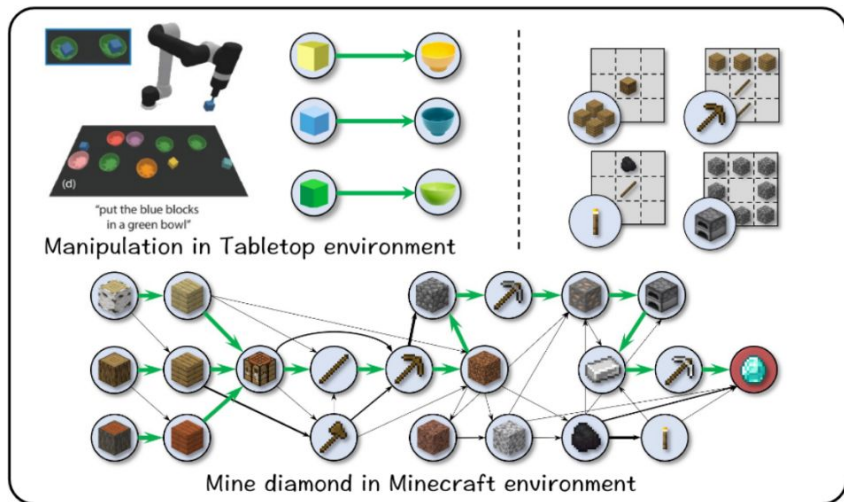


Open-world Planning

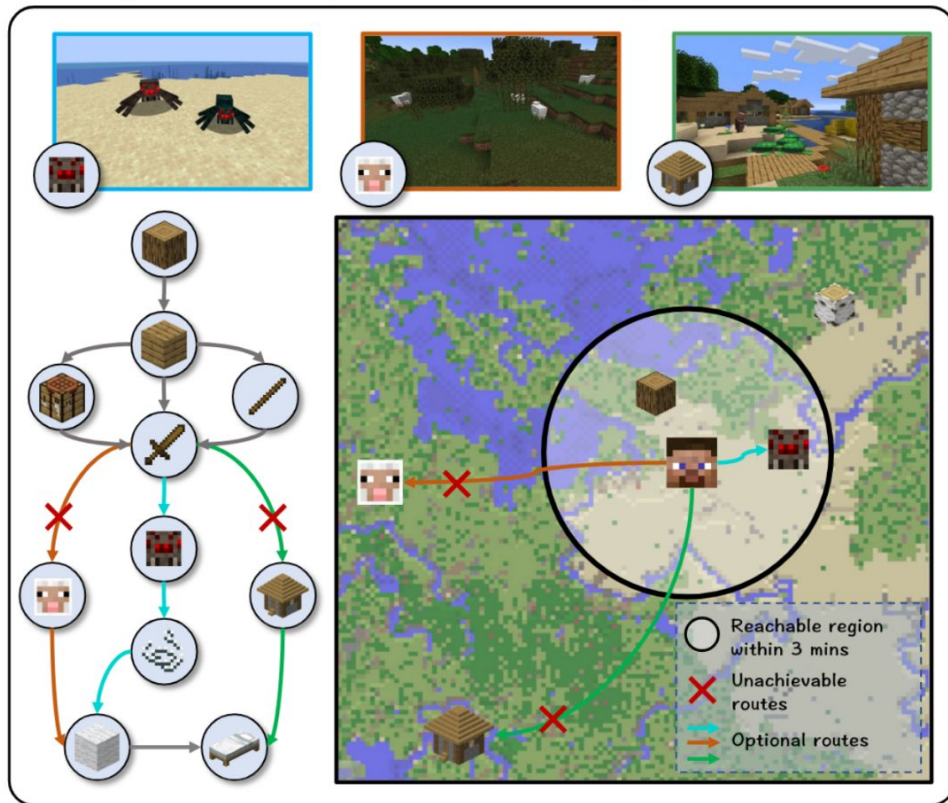
Planning success plummet in open worlds due to new challenges



Challenge #1: Complex Sub-task Dependency



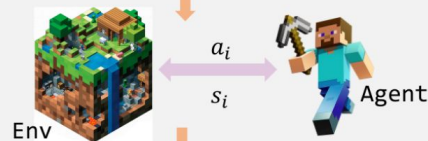
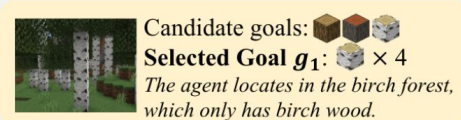
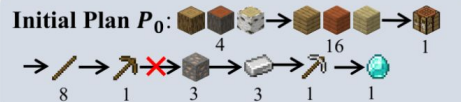
Challenge #2: State-dependent Task Feasibility



Open-world Planning

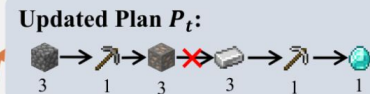
- we **can't benchmark** planning on fixed ground truth sequences in open-world
- running in simulator and measuring **success rate (SR)** is the only option
- the planning module has to be adaptive and be able to **modify the plan on the fly**

Task instruction: Obtain a diamond  in Minecraft survival mode step-by-step?



Description d_t : I **succeed** on goal 1-5. I **fail** on goal 6, mining 3  with . Now my inventory has 5 planks, ...

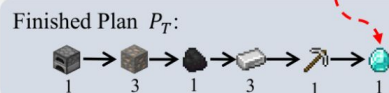
Explanation: Because mining  needs to use at least , which I do not have. So I need to craft  first.



Description d_t : I **succeed** on goal I **fail** on smelting 3  from 3 , on . My inventory now has 3 iron ore, ...

Explanation: Because smelting  needs to use  and , which I do not have. So I need to craft  first.

Task Finished !



ReplanVLM

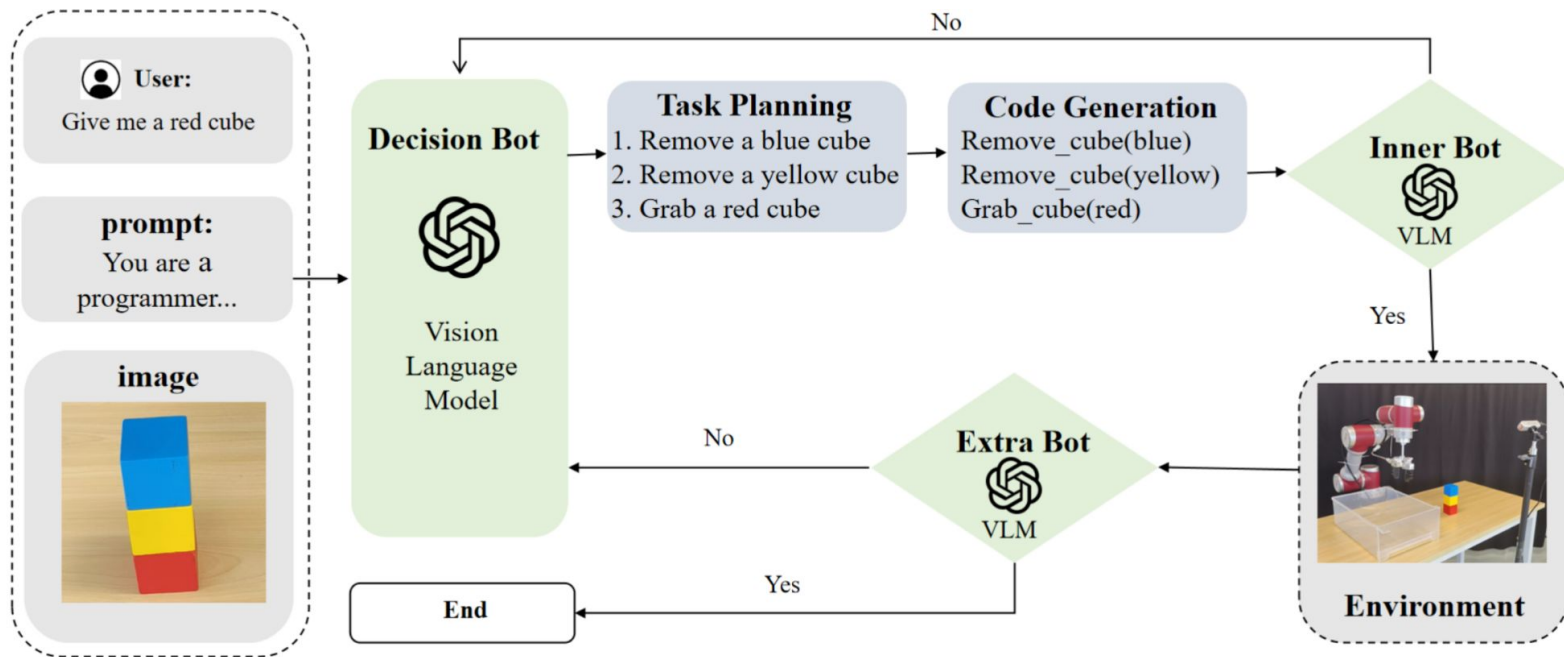
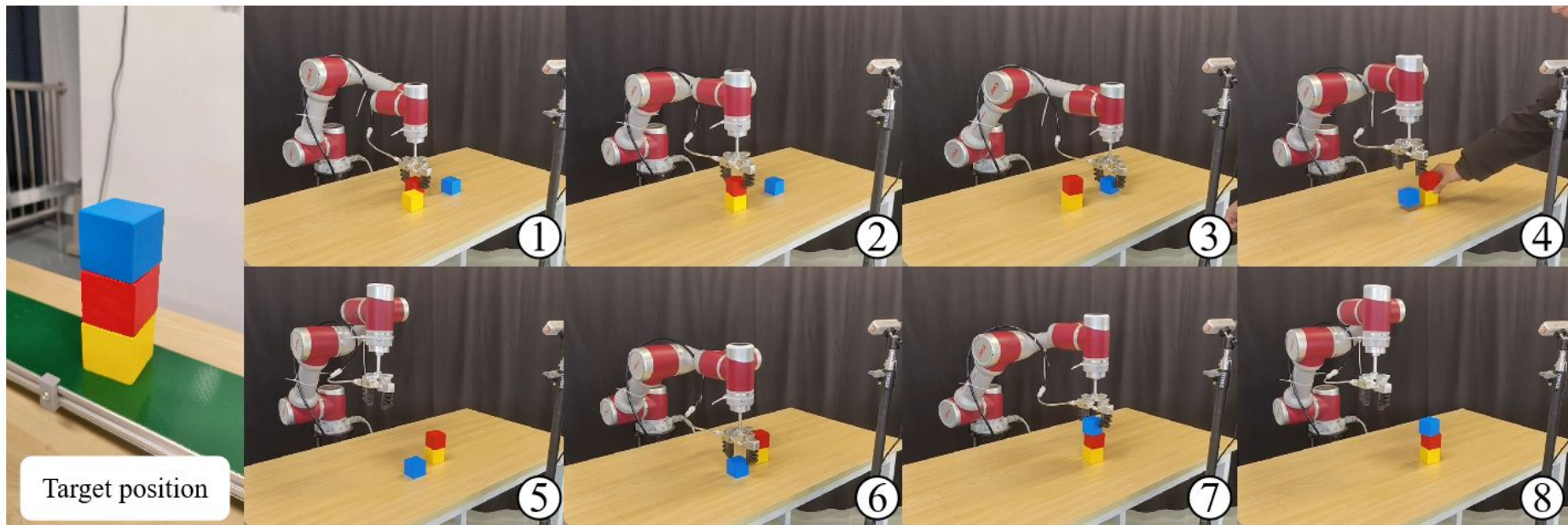


Fig. 1. ReplanVLM overview. We propose a ReplanVLM framework, which is composed of an internal error correction mechanism and an external error correction mechanism. The internal error correction mechanism refers to an Inner Bot, which is developed to assess and correct the task plans and codes generated by the Decision Bot. The external error correction mechanism involves an Extra Bot, which is used to determine whether the task is completed. If the task is deemed incomplete, the Extra Bot identifies the cause of the failure and sends it back to the Decision Bot to replan the task.

ReplanVLM



05

Acting: Manipulation

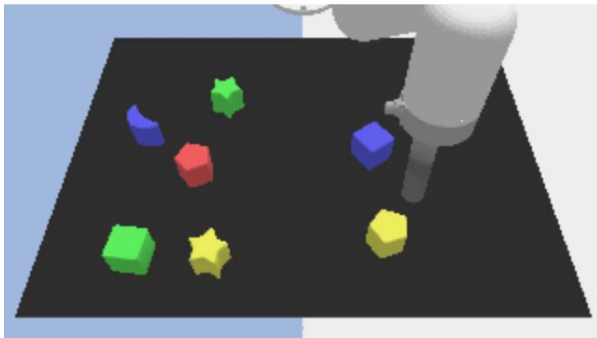


Manipulation

Given **language instruction** and **observations** from sensors, output **action/sequence of actions**. Modern notable approaches are based on:

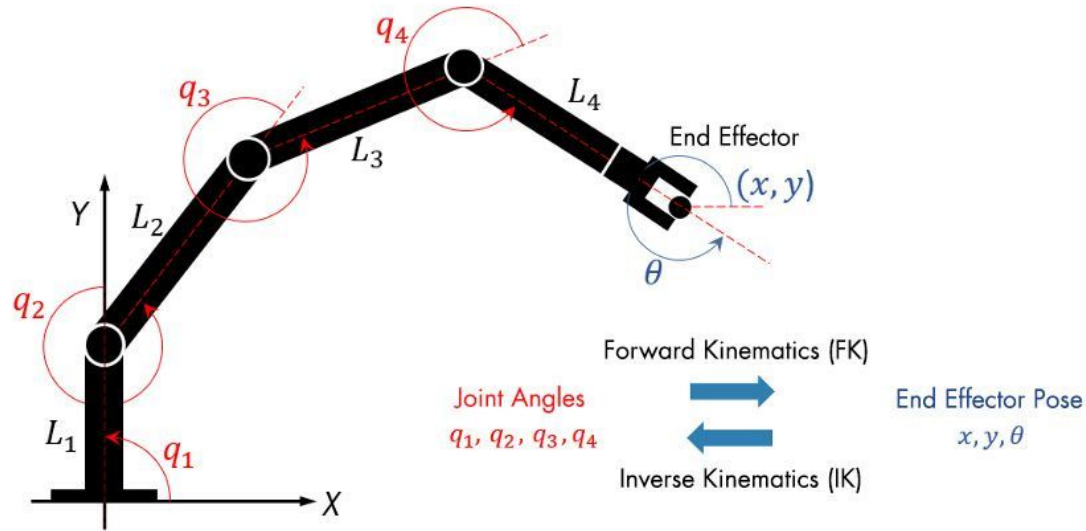
- transformers
- diffusion models
- hybrid models
- VLA

instruction: slide the green star next to the red moon

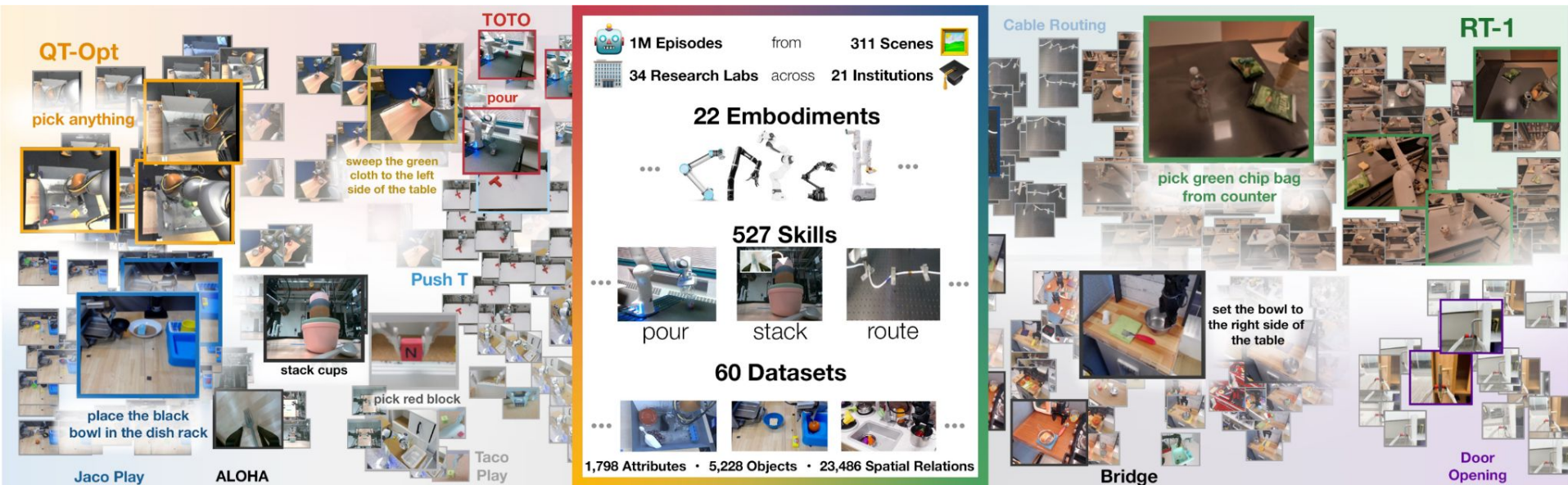


Manipulation: Inverse Kinematics

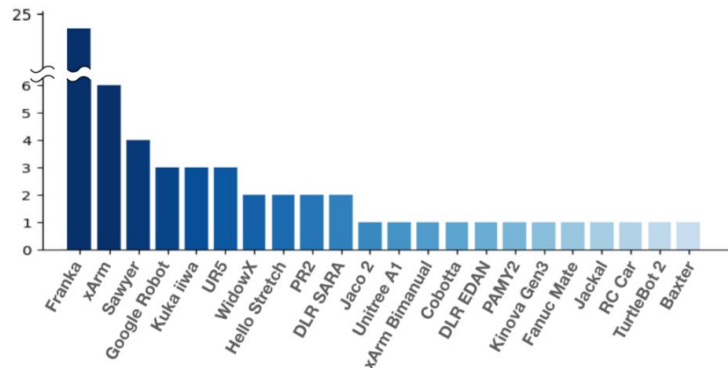
How the rest of the joints move is the **inverse kinematics** that gives out these joints



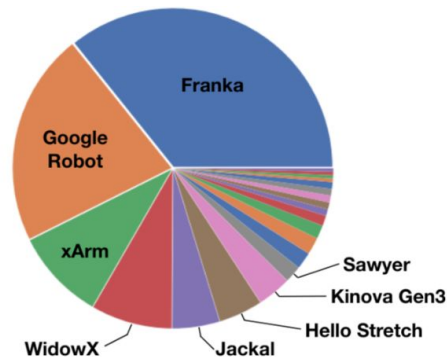
Towards ImageNet for Manipulation: Open X-Embodiment



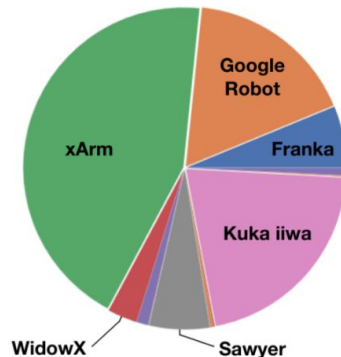
Open X-Embodiment



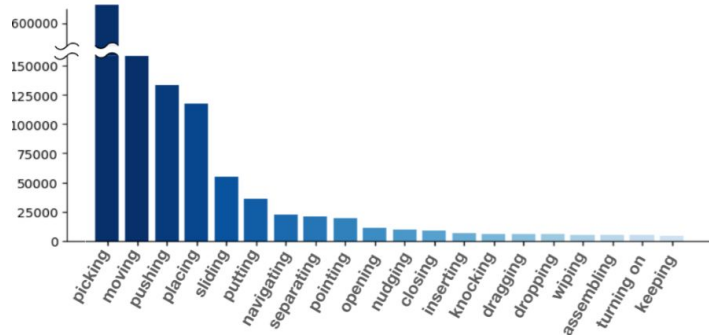
(a) # Datasets per Robot Embodiment



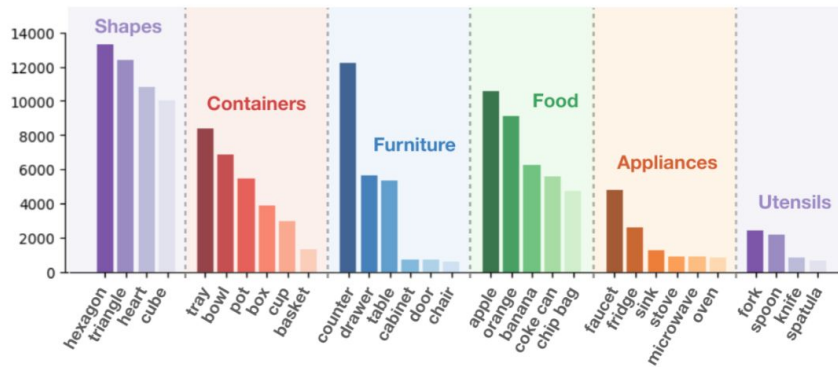
(b) # Scenes per Embodiment



(c) # Trajectories per Embodiment



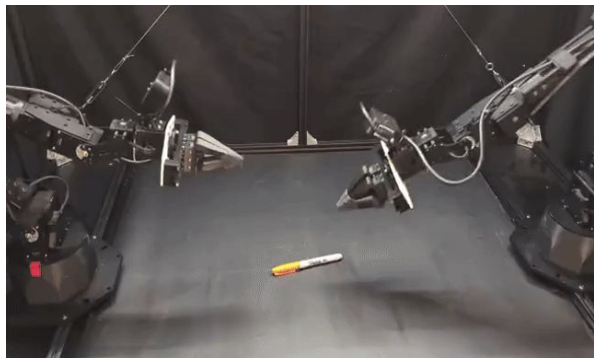
(d) Common Dataset Skills



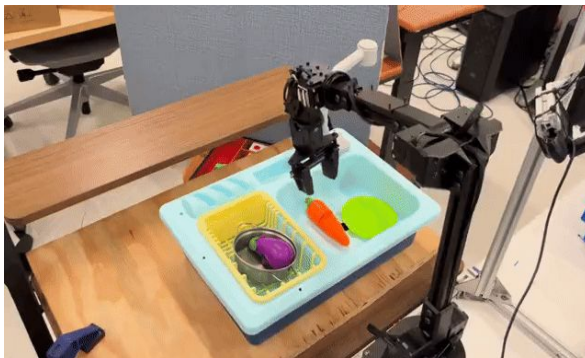
(e) Common Dataset Objects

Open X-Embodiment

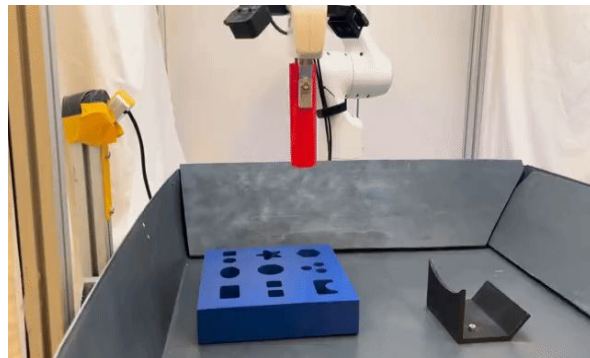
Examples of collected trajectories (22 embodiments)



dual-arm robot, Stanford
Aloha



part of the Bridge Dataset
WidowX BridgeV2



Functional manipulation benchmark
Berkley Peg insertion

Collecting Real Data: ALOHA

ALOHA 2 👍



Stanford
University



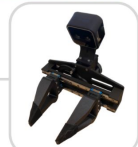
overhead camera



worms-eye camera



gravity
compensation



follower gripper



leader gripper

Collecting Real Data: ALOHA

Instead of the robot acting on its own, a person—sometimes sitting at a **console**, sometimes wearing a **VR headset** or **data-glove**

- **EgoView (first person camera)**
- **Separate navigation and manipulation**

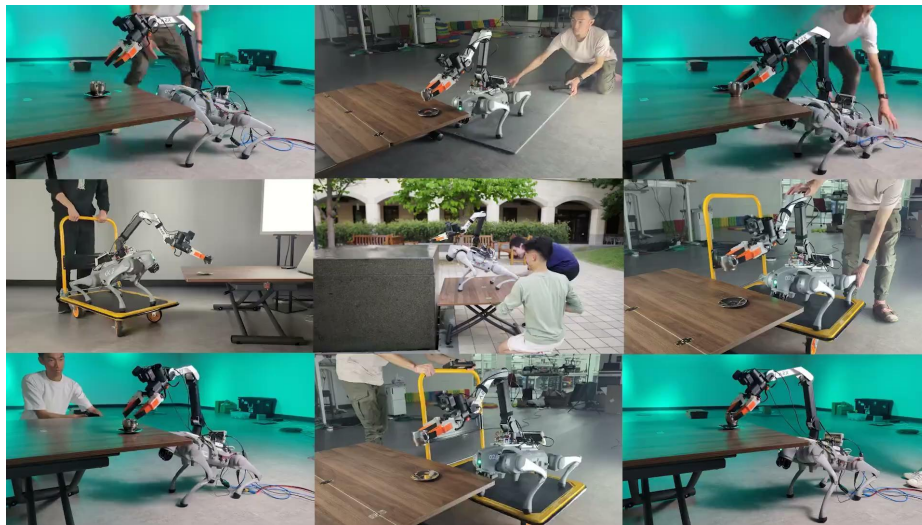
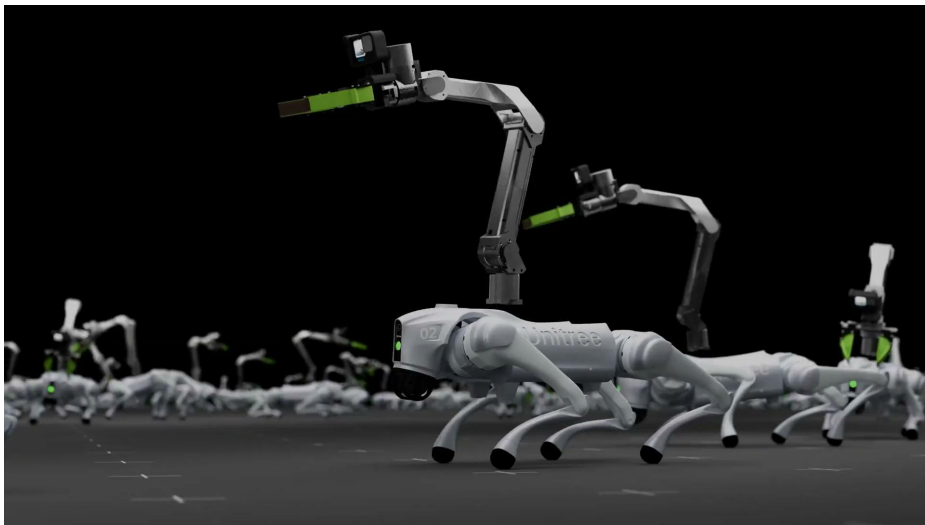


Collecting Real Data: UMI



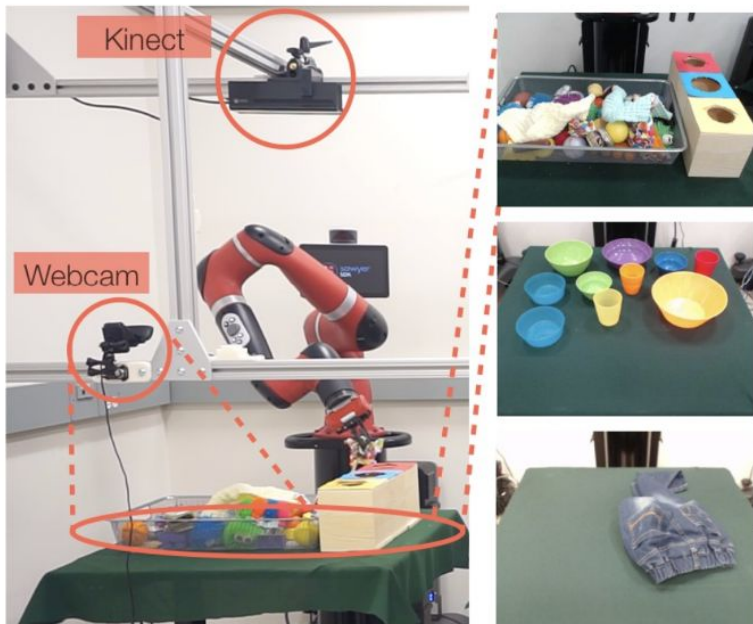
Adapting Real Data for Movement: UMI on Legs

A bunch of dogs are learning how to stabilize their hand using reward in a simulation



Collecting Real Data: CrowdSourcing

Start



Teleoperation across the world with RoboTurk



RoboTurk in the Swiss Alps
at ~7000 ft



Real-Time Teleoperation from Macau
to Stanford – over 11,000 km!

Transformer-based: Octo

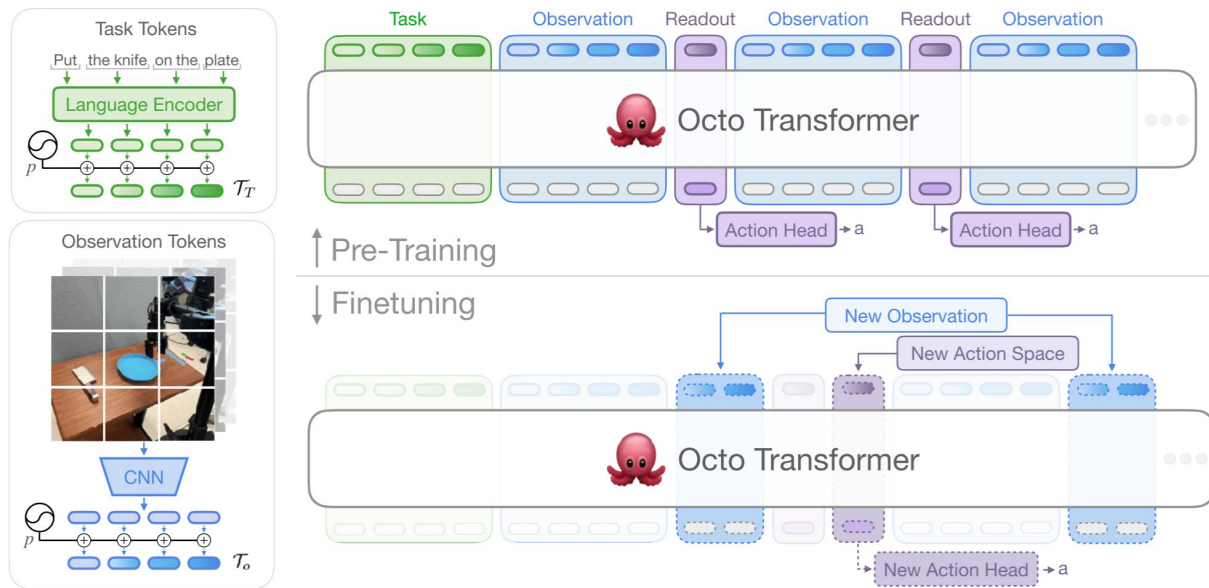


Fig. 2: **Model architecture.** **Left:** Octo tokenizes task descriptions (green) and input observations (blue) using a pretrained language model and a lightweight CNN, respectively. **Top:** The transformer backbone processes the sequence of task and observation tokens and produces readout tokens (purple) that get passed to output heads to produce actions. **Bottom:** The block-wise attention structure of the transformer allows us to add or remove inputs and outputs during finetuning: for example, we can add new observations (blue, dashed) or action spaces (purple, dashed) without modifying any pretrained parameters.

3 components:
tokenizer, transformer, action head

t5-base (111M) for text tokeniz.
Octo-Small (27M) $\approx$ ViT-S
Octo-Base (93M) $\approx$ ViT-B

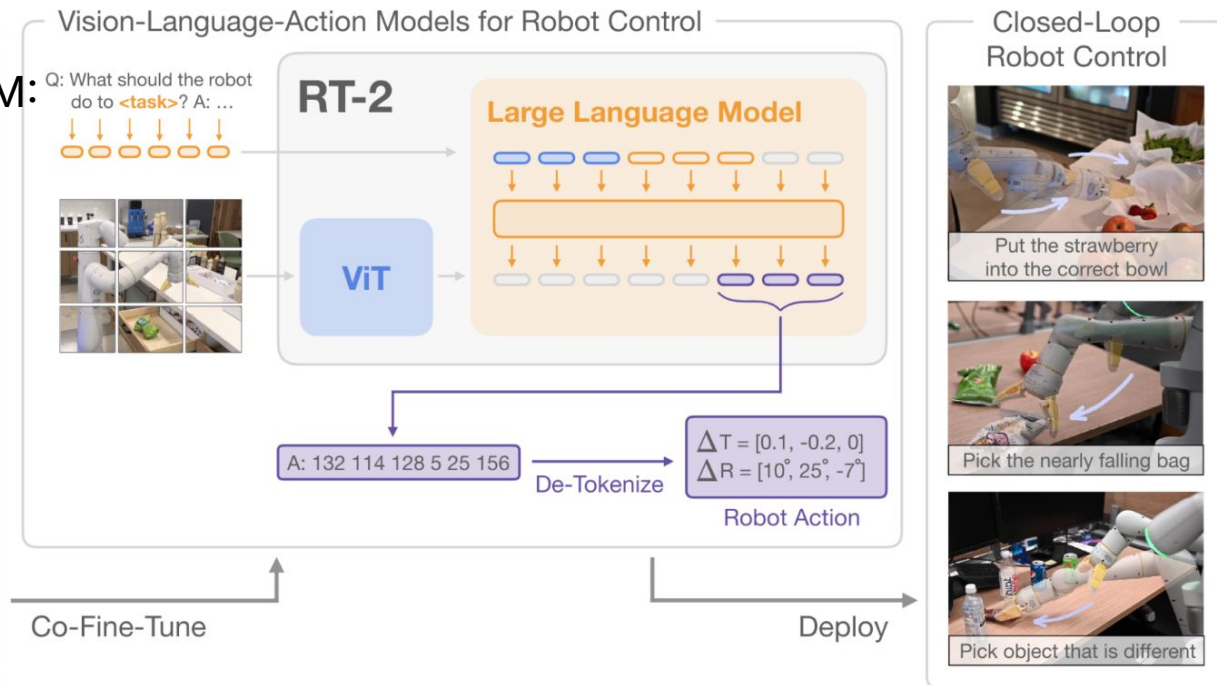
Pretrained on 128 TPUV4
($\approx$ 200 A100) for 14 hours,
much longer than on ImageNet

May be finetuned
on 3090 in 4 hours

Good starting point
for training your own
manipulation policies

VLLMs as Policies: RT-2

- **Reusing knowledge** from LLM: easier generalizability
- Manipulating objects that it **has not seen**
- Action questions + VQA for avoiding **catastrophic forgetting**



VLLMs as Policies: RT-2

Aim to utilize general knowledge obtained from web data

Internet-Scale VQA + Robot Action Data



Q: What is happening in the image?

A: 311 423 170 55 244

A grey donkey walks down the street.

Q: Que puis-je faire avec ces objets?

A: 3455 1144 189 25673

Faire cuire un gâteau.



Q: What should the robot do to <task>?

A: 132 114 128 5 25 156

$\Delta \text{Translation} = [0.1, -0.2, 0]$

$\Delta \text{Rotation} = [10^\circ, 25^\circ, -7^\circ]$

Vision-Language-Action Models for Robot Control

Q: What should the robot do to <task>? A: ...



RT-2

ViT

Large Language Model

A: 132 114 128 5 25 156

De-Tokenize

$\Delta T = [0.1, -0.2, 0]$
 $\Delta R = [10^\circ, 25^\circ, -7^\circ]$

Robot Action

Co-Fine-Tune

Deploy

Closed-Loop Robot Control

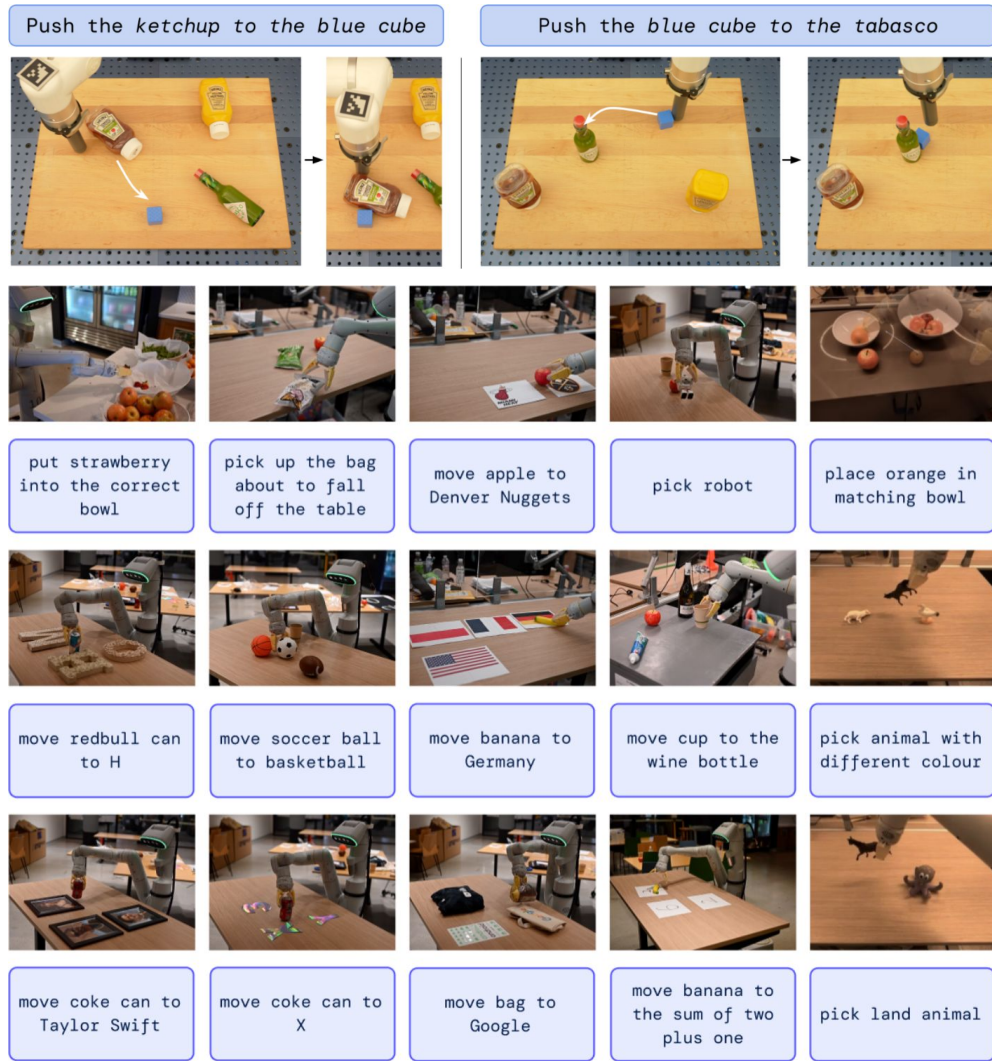


VLLMs as Policies: RT-2

Key idea: reuse knowledge from LLM

Results:

- better generalization to unseen objects and environments
- symbol understanding:
move apple to 3
- reasoning:
move apple to cup with same color, move X near the sum of two plus one, mueve la manzana al vaso verde
- human recognition:
move the coke can to the person with glasses



RT-2 Simple Chain of Thought

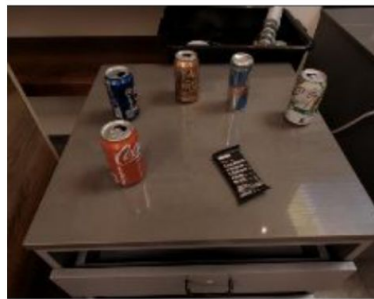
Prompt:
Given Instruction:
Bring me a drink.
Prediction:
Plan: pick 7up can.
Action: 1 143 129 123 145
114 115 127



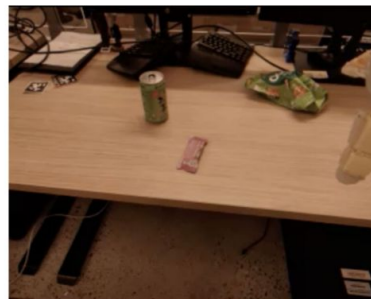
Prompt:
Given Instruction:
Move all the objects
together.
Prediction:
Plan: move green can near
green rice chip bag.
Action: 1 128 126 127 135
123 119 127



Prompt:
Given Instruction:
Pick the object that is
different from all other
objects
Prediction:
Plan: pick rxbar
chocolate. Action: 1 128
129 125 131 125 128 127



Prompt:
Given Instruction:
Move the green objects
together.
Prediction:
Plan: move green can near
green rice chip bag.
Action: 1 130 129 121 131
127 128 127

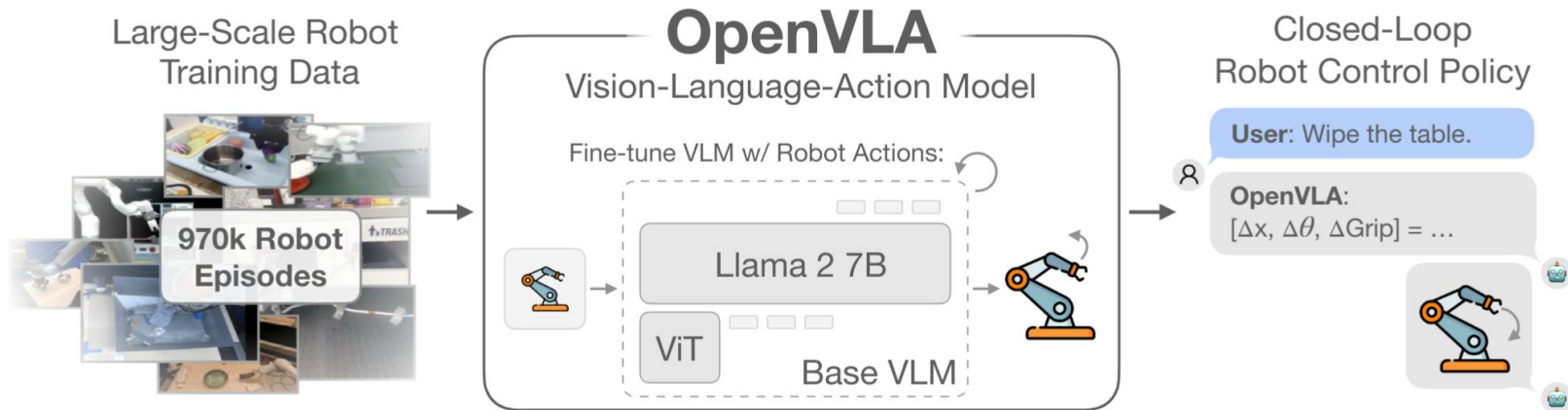


Prompt:
Given I need to
hammer a nail, what
object from the scene
might be useful?
Prediction:
Rocks. Action: 1 129 138
122 132 135 106 127



Figure 7 | Rollouts of RT-2 with chain-of-thought reasoning, where RT-2 generates both a plan and an action.

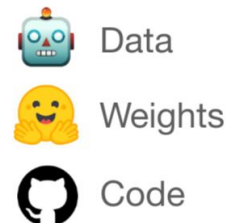
VLLMs as Policies: OpenVLA



Multi-Robot Control & Efficient Fine-Tuning



Fully Open-Source



OpenVLA

Pretrained on 64 A100 for 14 days, finetuned on 8 A100 for 5-15 hours
Runs at 6Hz on 4090 in bfloat16 format

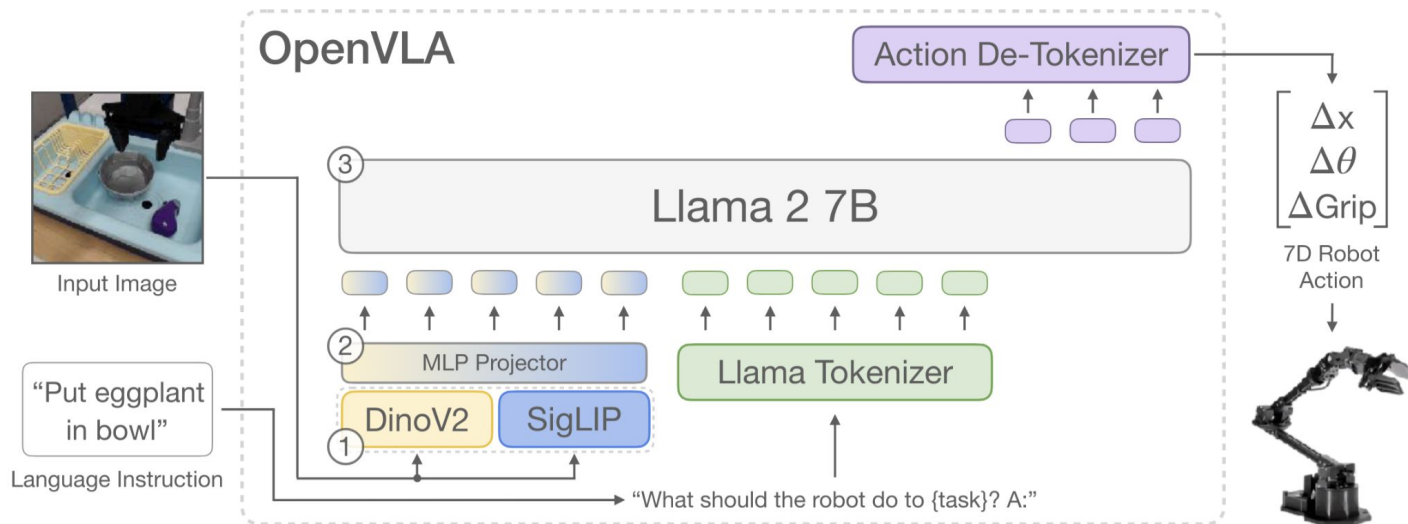


Figure 2: **OpenVLA model architecture.** Given an image observation and a language instruction, the model predicts 7-dimensional robot control actions. The architecture consists of three key components: (1) a **vision encoder** that concatenates Dino V2 [25] and SigLIP [77] features, (2) a **projector** that maps visual features to the language embedding space, and (3) the **LLM backbone**, a Llama 2 7B-parameter large language model [10].

OpenVLA

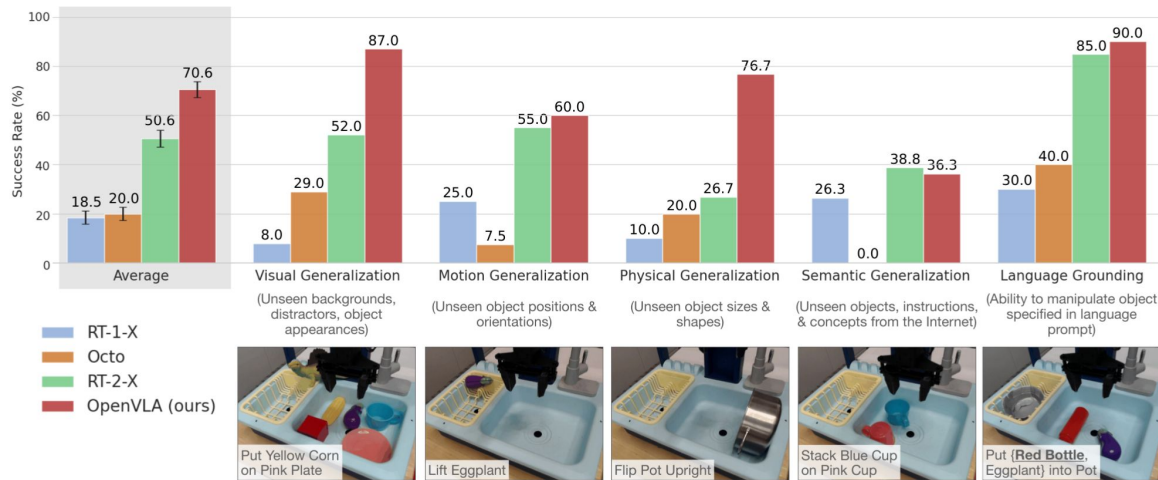


Figure 3: **BridgeData V2 WidowX robot evaluation tasks and results.** We evaluate OpenVLA and prior state-of-the-art generalist robot policies on a comprehensive suite of tasks covering several axes of generalization, as well as tasks that specifically assess language conditioning ability. OpenVLA achieves highest overall performance and even outperforms closed-source model RT-2-X in all categories except for semantic generalization. Average success rates $\pm$ StdErr are computed across 170 total rollouts per approach. See Table 4 for detailed results.

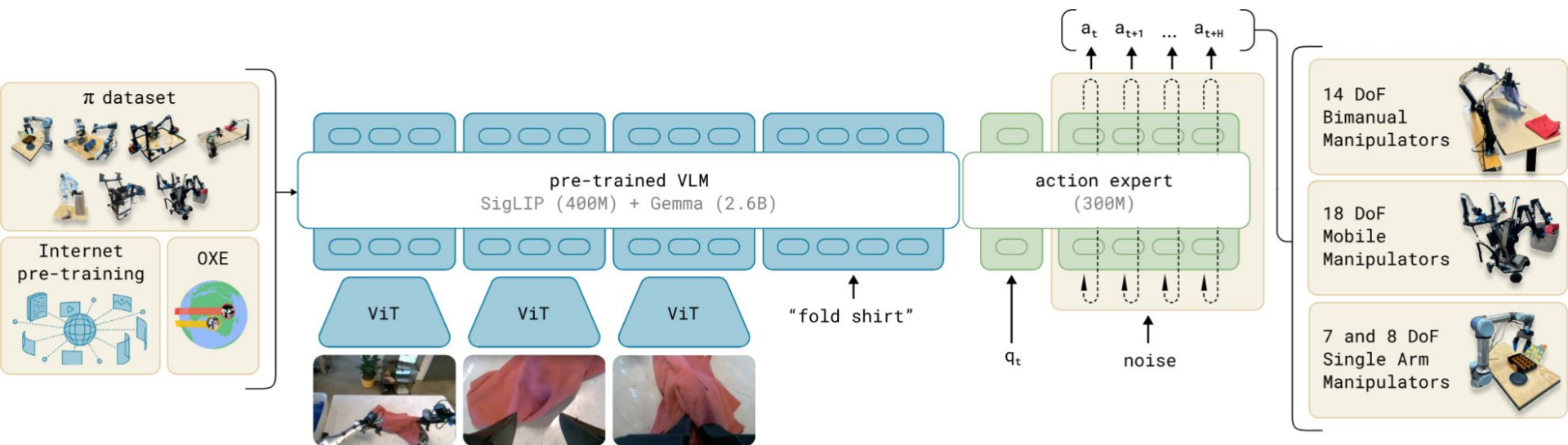
Table 1: **Parameter-efficient fine-tuning evaluation.** LoRA fine-tuning [26] achieves the best performance-compute trade-off, matching full fine-tuning performance while training only 1.4% of the model parameters. Mean success rate $\pm$ StdErr is computed across 33 rollouts per approach on select Franka-Tabletop tasks.

*: Sharded across 2 GPUs with FSDP [75].

Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	69.7 ± 7.2 %	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	68.2 ± 7.5 %	97.6	59.7 GB
rank=64	68.2 ± 7.8 %	195.2	60.5 GB

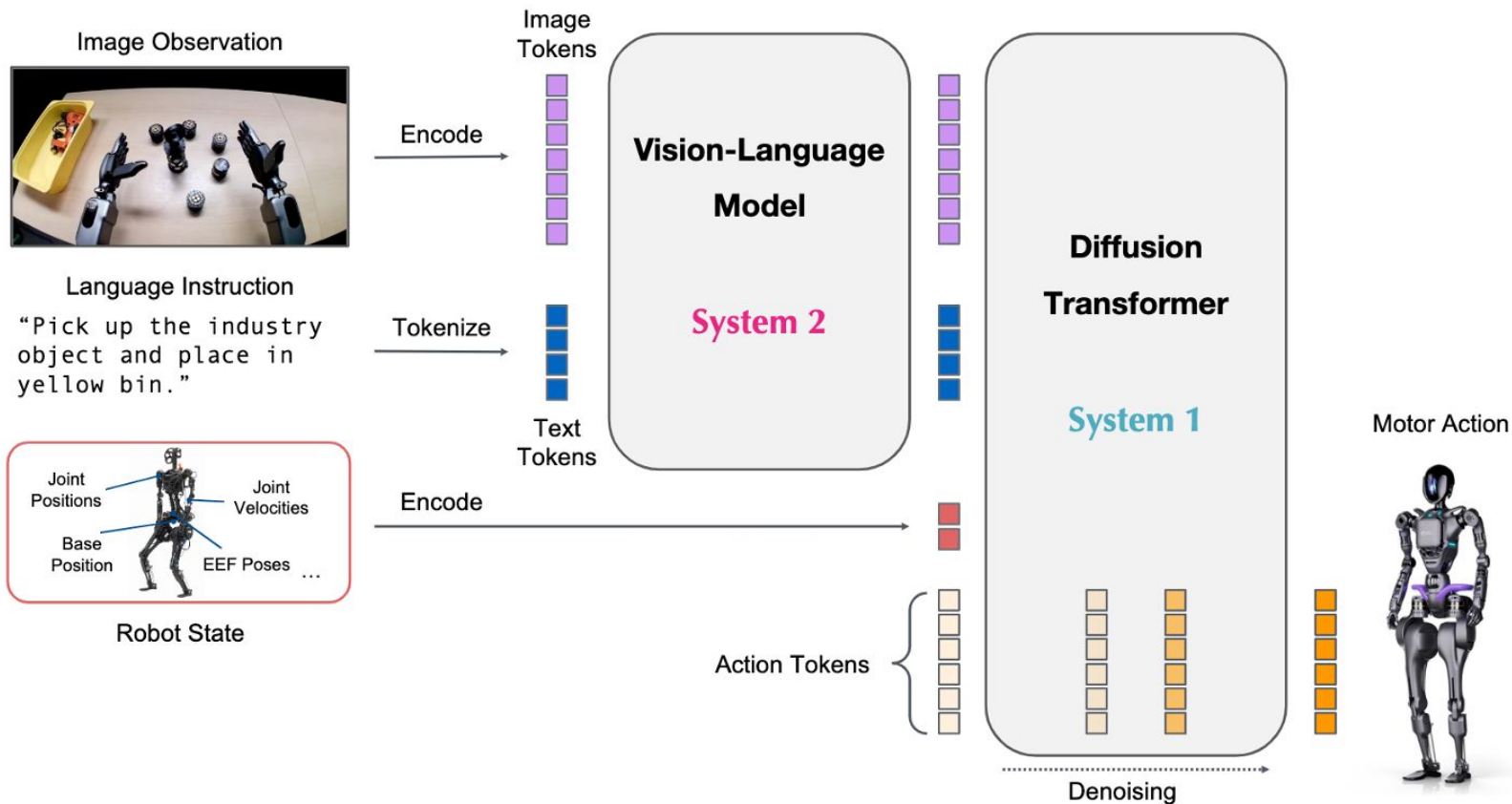
8x speedup when training with LoRA (we got $\pm 3x$)

Pi 0/0.5

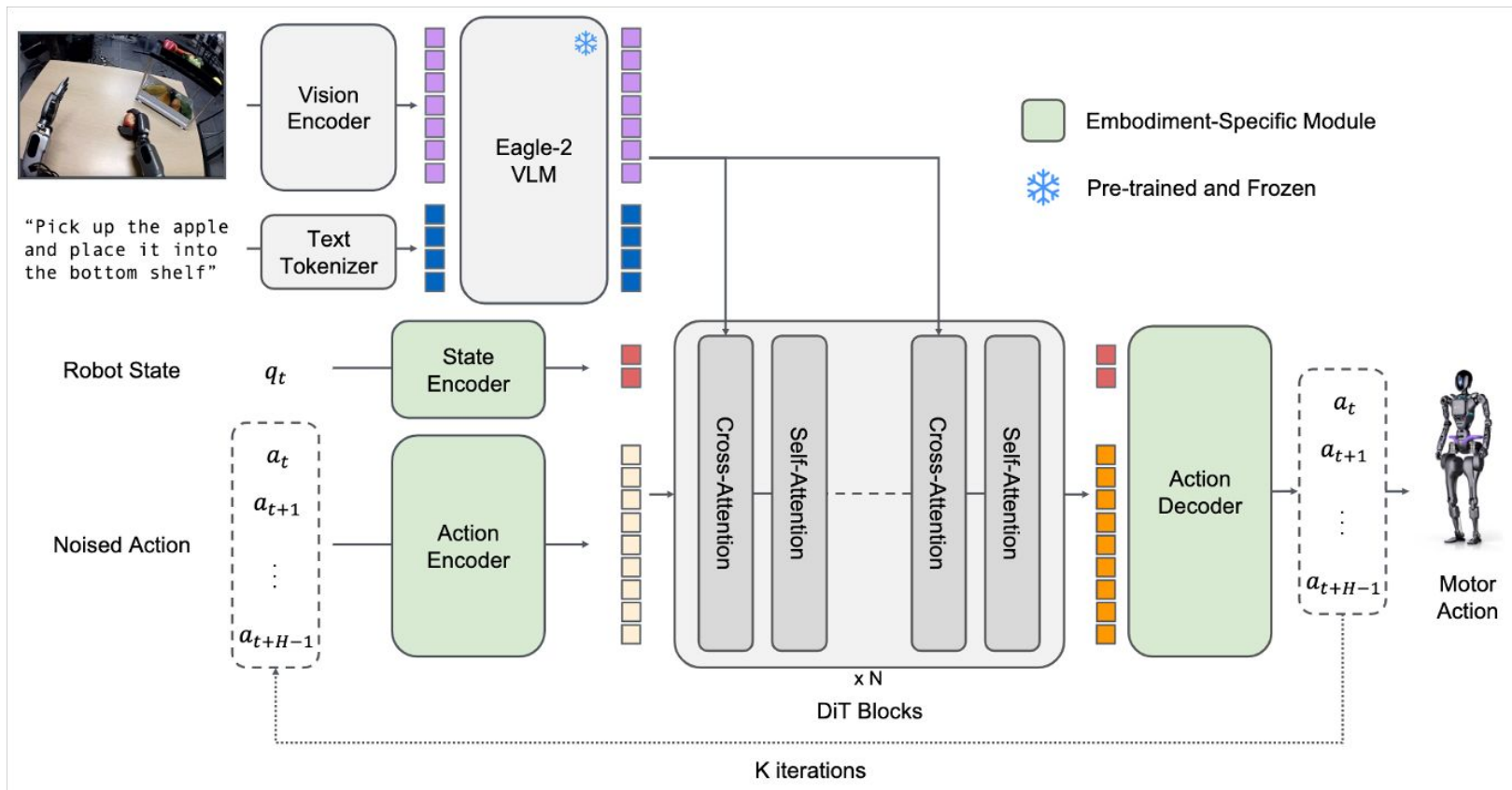


- PaliGemma VLM backbone
- Co-training with multimodal web data
- 10x various (envs, embodiments) action data for pretraining
- Action expert (diffusion/flow matching head) on top of VLM for precise manipulation

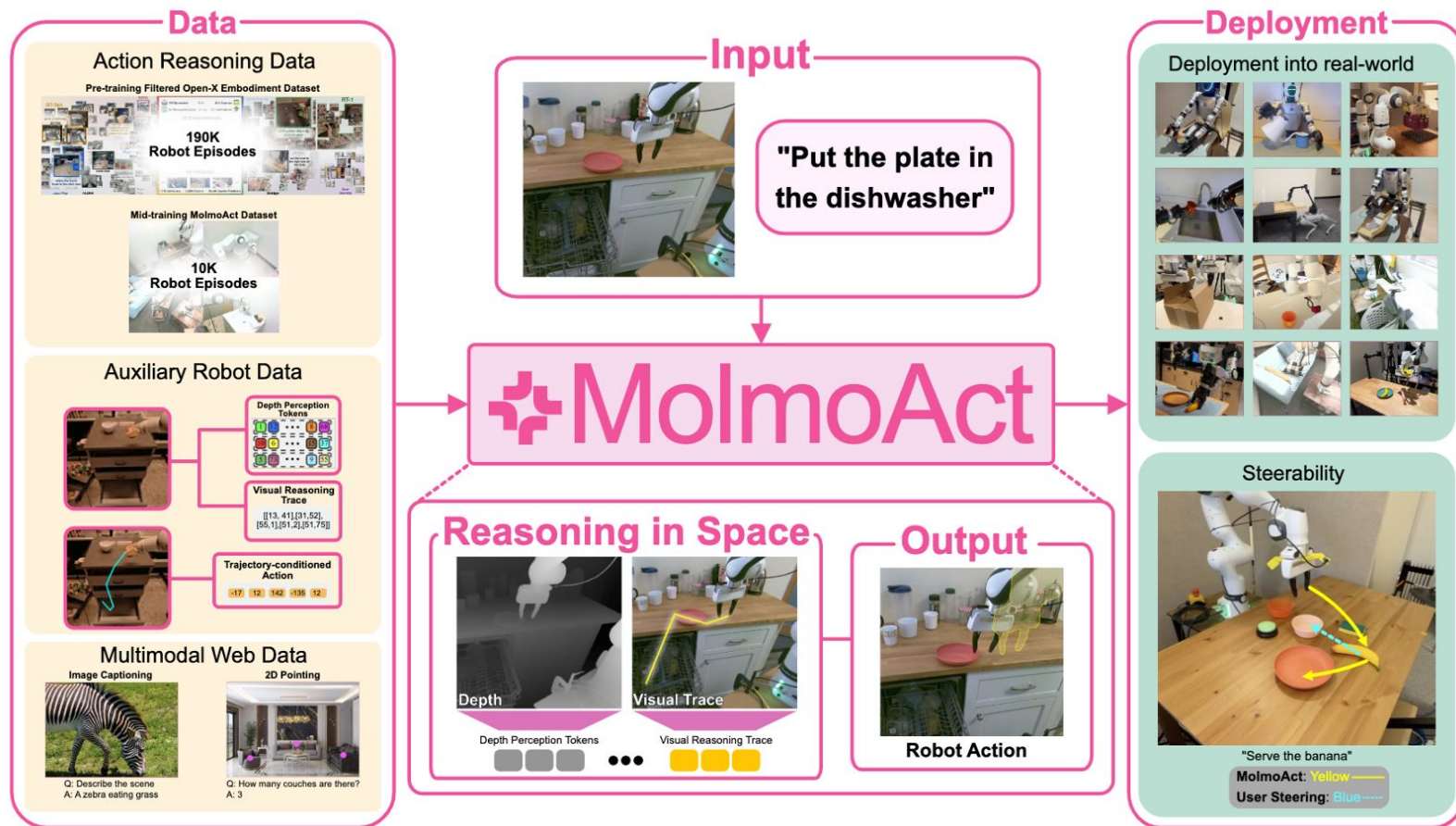
GR00T N1



GR00T N1



MolmoAct: Spatial Reasoning Before Acting



MolmoAct: Spatial Reasoning Before Acting



The task is pick water bottle from middle drawer and place on counter. Notice that the trajectory of the end effector is annotated on the image. Based on the trajectory annotated on the image, what is the action that the robot should take?

Based on the image and the annotated trajectory of the end effector, the action that the robot should take is [0í, áY±, īōh, ā¶ju, ā, h, ôl;ji, ôêlij].



The task is pick blue chip bag from bottom drawer and place on counter. Notice that the trajectory of the end effector is annotated on the image. Based on the trajectory annotated on the image, what is the action that the robot should take?,"

Based on the image and the annotated trajectory of the end effector, the action that the robot should take is [âKl, ī'ī, ôêlij, ā½, ā½L, īā; j, Y¥].

The task is Place the spoon on the lower left table edge. What is the action that the robot should take. To figure out the action that the robot should take to Place the spoon on the lower left table edge, let's think through it step by step. First, what is the depth map for this image? Second, what is the trajectory of the end effector? Based on the depth map of the image and the trajectory of the end effector, what is the action that the robot should take?



The depth map of the image is

<DEPTH_START><DEPTH_75><DEPTH_101><DEPTH_69><DEPTH_8><DEPTH_69><DEPTH_56><DEPTH_56><DEPTH_30><DEPTH_32><DEPTH_101><DEPTH_26><DEPTH_56><DEPTH_32><DEPTH_116><DEPTH_56><DEPTH_101><DEPTH_32><DEPTH_69><DEPTH_8><DEPTH_75><DEPTH_32><DEPTH_116><DEPTH_9><DEPTH_9><DEPTH_116><DEPTH_32><DEPTH_32><DEPTH_32><DEPTH_69><DEPTH_101><DEPTH_75><DEPTH_116><DEPTH_69><DEPTH_9><DEPTH_9><DEPTH_9><DEPTH_69><DEPTH_9><DEPTH_101><DEPTH_69><DEPTH_69><DEPTH_69><DEPTH_69><DEPTH_69><DEPTH_69><DEPTH_32><DEPTH_69><DEPTH_69><DEPTH_32><DEPTH_56><DEPTH_56><DEPTH_8><DEPTH_69><DEPTH_69><DEPTH_32><DEPTH_56><DEPTH_32><DEPTH_9><DEPTH_56><DEPTH_56><DEPTH_56><DEPTH_56><DEPTH_8><DEPTH_32><DEPTH_32><DEPTH_32><DEPTH_56><DEPTH_69><DEPTH_101><DEPTH_56><DEPTH_69><DEPTH_32><DEPTH_101><DEPTH_8><DEPTH_8><DEPTH_56><DEPTH_101><DEPTH_69><DEPTH_69><DEPTH_75><DEPTH_101><DEPTH_69><DEPTH_25><DEPTH_74><DEPTH_56><DEPTH_56><DEPTH_32><DEPTH_56><DEPTH_101><DEPTH_26><DEPTH_26><DEPTH_25><DEPTH_76><DEPTH_76><DEPTH_76><DEPTH_103><DEPTH_69><DEPTH_25><DEPTH_74><DEPTH_116><DEPTH_116><DEPTH_END>. The trajectory of the end effector is [[30,39],[56,81],[58,70]]. Based on these information, the action that the robot should take is [āJ, ākl, ôijāK, ī'h, ē±j], āēlj, ôêlij].

MolmoAct: Spatial Reasoning Before Acting

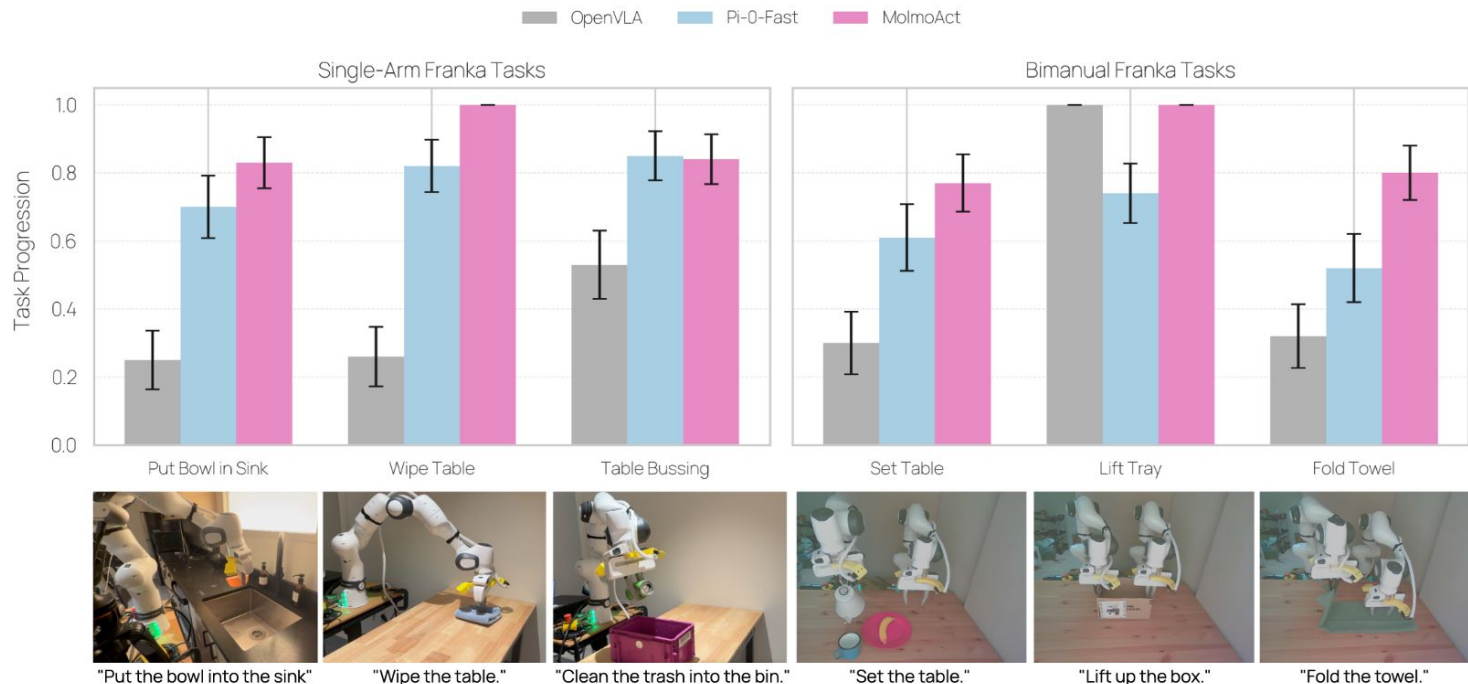


Figure 5 Real-world evaluation of OpenVLA, π_0 -FAST, and MOLMOACT on single-arm (left) and bimanual (right) Franka tasks. Bar plots report average task progression with standard error across 25 trials per task. MOLMOACT consistently outperforms baselines, particularly on single-arm tasks such as *Wipe Table* and *Table Bussing*, and maintains strong performance on bimanual tasks including *Fold Towel* and *Set Table*. Bottom row shows example task setups with corresponding natural language instructions.

Open questions for VLAs

Future research directions:

- VLA with long context for multiple video frames, memory, more modalities
- Strong simulators with diverse tasks to objectively assess VLAs performance
- Real-time inference (60+Hz) on device.
Now most models runs $< 20\text{Hz}$ even with low resolution
- Performance improvement (now SR $< 90\%$)
- Cross-embodied generalizability
- Co-training for VQA and action prediction is to be explored
- Increasing robustness to text and image perturbations

06

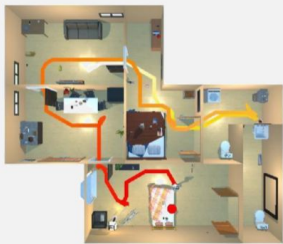
Acting: navigation



SPOC

SIMULATOR

"Navigate to a basketball"



"Locate a laptop"



"Fetch a mug"



SPOC's manipulation camera view while picking up the mug

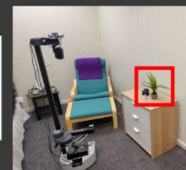
"Navigate to a house plant"



SPOC begins in the living room



SPOC explores 3 rooms and a hallway

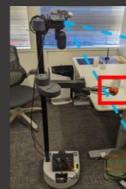


SPOC locates the house plant and navigates towards it

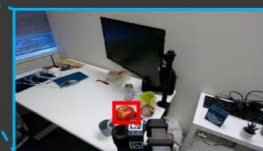
"Fetch an apple"



SPOC explores until it sees the apple



SPOC picks up the apple



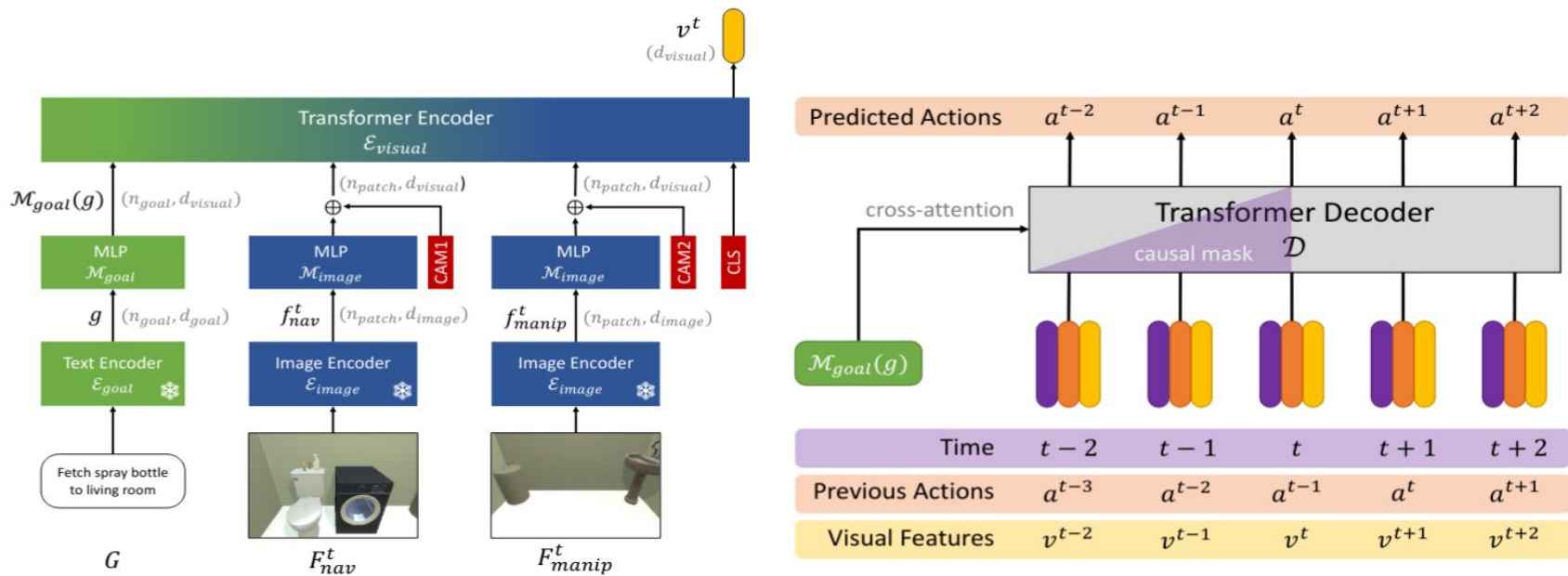
SPOC's manipulation camera view while picking up the apple

REAL WORLD



SPOC

Discr. actions: move base ($\pm 20\text{cm}$), rotate base ($\pm 6^\circ, \pm 30^\circ$), move arm (x, z) ($\pm 2\text{cm}, \pm 10\text{cm}$)



SPOC

Data:

- AI2THOR simulator
- 40k household objects from Objaverse dataset
- Use ProcTHOR to procedurally generate 200k houses (1-8 rooms)

Trajectories:

- Navigation: go to target using approximation of shortest path
- Manipulation: navigate to object, then iteratively minimize distance between robot and object
- Room visitation: calculate center of house, then navigate to all rooms via shortest paths



SPOC Benchmarks

CHORES:

- navigation
- object recognition
- object manipulation
- exploration

CHORES Nav:

- open voc. instruction following
- object affordance
- scene understanding
- object comparison

Task	Description & Example
OBJNAV	Locate an object category: “find a mug”
PICKUP	Pick up a specified object in agent line of sight: “pick up a mug”
FETCH	Find and pick up an object: “locate a mug and pick up that mug”
ROOMVISIT	Traverse the house. “Visit every room in this 5-room house. Indicate when you have seen a new room and when you are done.”

Table 1. CHORES tasks.

Task	Target Description & Example
OBJNAV	Object’s category: “vase”
OBJNAVAFFORD	Object’s possible uses: “a container that can best be used for holding fresh flowers decoratively”
OBJNAVLOCALREF	Object’s nearby objects: “a vase near a tennis racket and a basketball”
OBJNAVRELATTR	Object category comparative attribute: “the smallest vase in the bedroom”
OBJNAVROOM	Object’s room type: “vase in the living room”
OBJNAVDESC	Open vocab instance description: “the brown vase painted orange with a bird on the side”
ROOMNAV	Type of room: “bedroom”

Table 2. CHORESNAV tasks. The full task specification also includes a navigation verb, such as “Search for a vase”.

SPOC

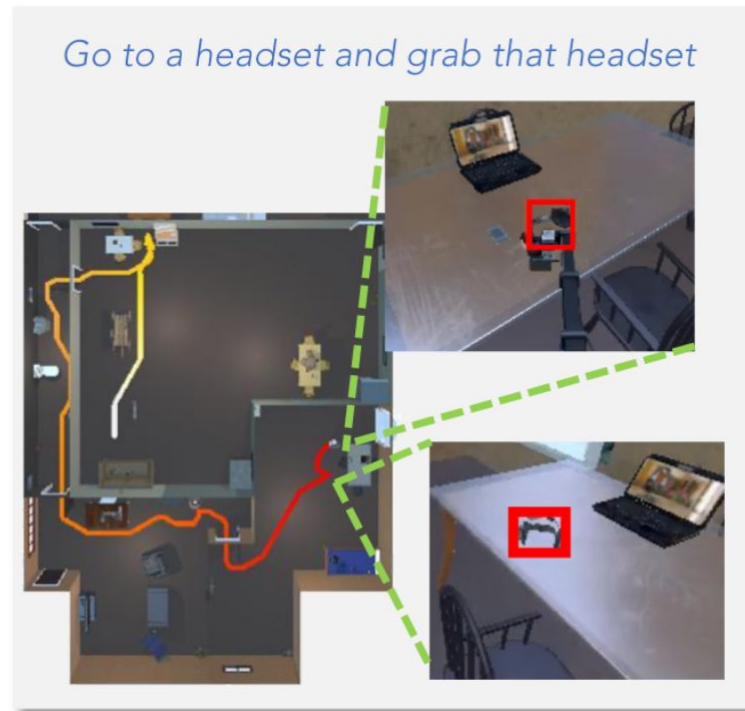
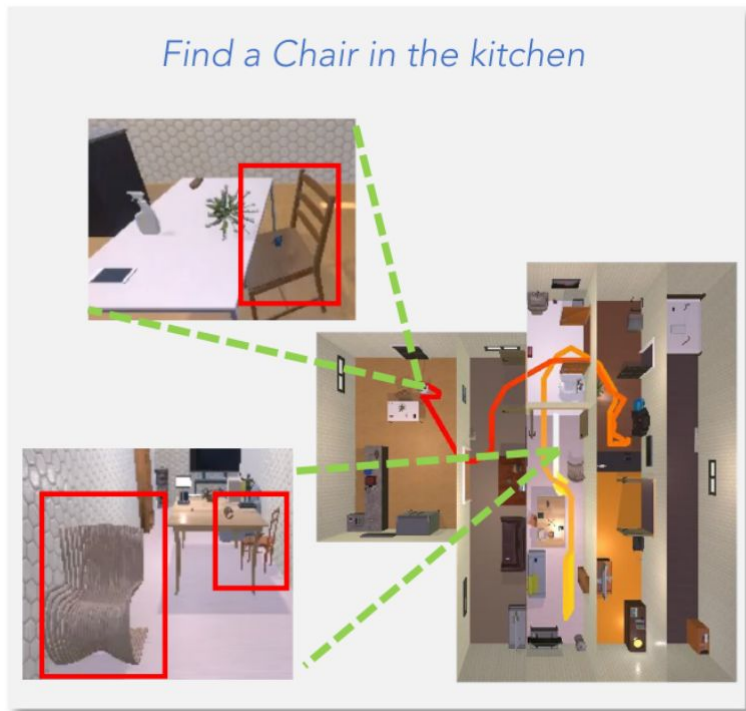
Key results:

- Imitation learning outperforms RL in speed (3500 FPS vs 175 FPS) and quality
- Architecture scales well to multiple tasks
- Object detection means a lot
- Transformer encoders and decoders boost quality a lot
- Long horizon tasks require long context windows (100 timesteps here vs 6 timesteps in RT-1)
- SPOC generalizes to real world

Benchmark	Model	Training	OBJNAV			PICKUP			FETCH			ROOMVISIT			Avg Success
			Success	SEL	%Rooms	Success	SEL	%Rooms	Success	SEL	%Rooms	Success	SEL	%Rooms	
CHORES -S	EmbSigLIP* [38]	Single-task RL	36.5	24.5	42.2	71.9	52.9	30.3	0.0	0.0	50.5	16.5	11.9	44.6	31.2
	SPOC-1-task	Single-task IL	57.0	46.2	51.5	84.2	81.0	30.3	15.1	12.6	48.1	43.7	40.4	81.2	50.0
	SPOC	Multi-task IL	55.0	42.2	56.3	90.1	86.9	30.3	14.0	10.5	49.3	40.5	35.7	81.1	49.9
	SPOC w/ GT Det	Multi-task IL	85.0	61.4	58.7	91.2	87.9	30.3	47.3	35.6	61.6	36.7	33.7	79.3	65.0
CHORES -L	SPOC	Multi-task IL	33.7	25.1	53.7	75.1	69.1	31.5	10.6	8.1	42.9	35.0	33.2	77.8	38.6
	SPOC w/ GT Det	Multi-task IL	83.9	58.0	64.0	78.0	75.7	31.5	48.6	38.3	60.0	42.0	39.1	83.1	63.1

Model	OBJNAV	PICKUP	FETCH	ROOMVISIT	Average
SPOC	50.0	46.7 (66.7)	11.1 (33.3)	50.0	39.5
SPOC w/ DETIC	83.3	46.7 (86.7)	44.4 (44.4)	50.0	56.1

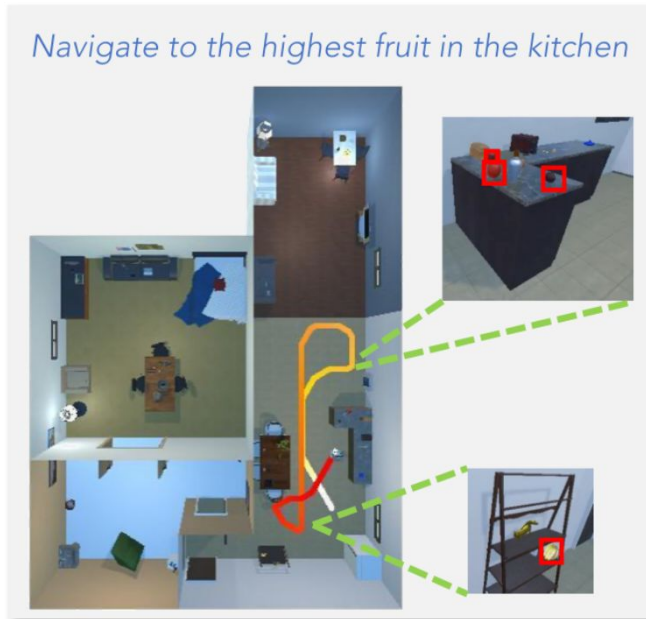
SPOC



Left: skipping all chairs to find one in the kitchen

Right: repositioning itself to find a location where headset is reachable

SPOC



Left: looking for all fruits in the kitchen and then navigating to highest

Right: looking for a sofa which has a laptop on it

07

Reward Functions

RoboCoT

Language Input



open the door

Chain-of-Thought



1. the robot hand approaches the door's handle
2. ...

Sparse Failure Guidance



the robot hand is stuck at the left and top of the door.

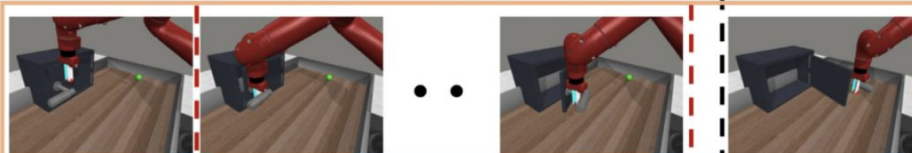


...



$$r_{i:i+T} = \frac{\max_{z_t \in \{z_t^+\}} \exp(z_v \cdot z_t / \tau)}{\max_{z_t \in \{z_t^+\}} \exp(z_v \cdot z_t / \tau) + \sum_{z'_t \in \{z_t^-\}} \exp(z_v \cdot z'_t / \tau)}$$

success



Reward Design

Analysis Template

Consider the following image:

Image: [Image 1]

this is a simulation environment. In this image, the door of the box is open. We define it when the door is open, the output is 1. Otherwise, the output is 0.

Labeling Template

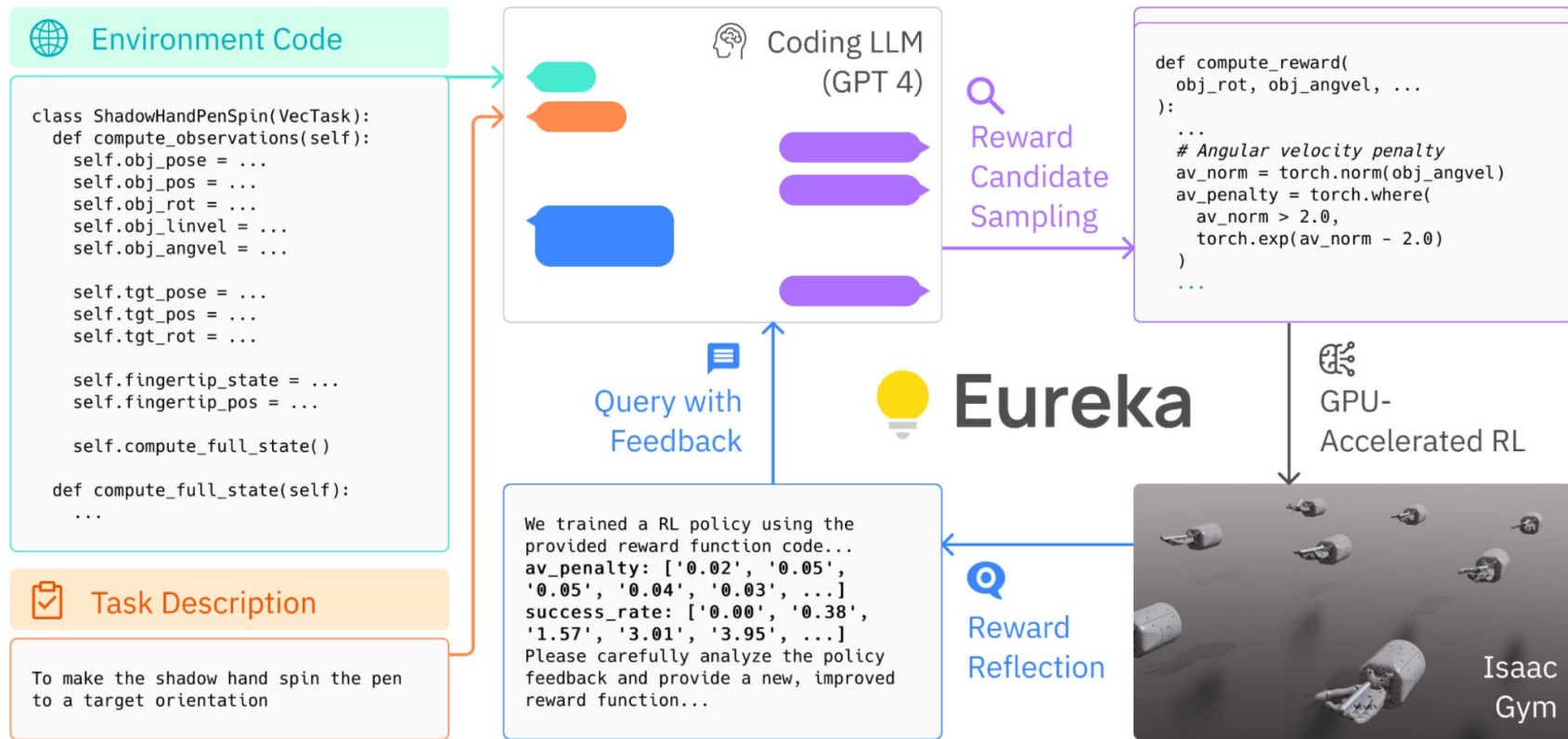


Image: [Final Observation]

What is the output of this image?

Self Imitation, i.e., Success Regularization

Eureka



Eureka

```
def compute_reward(object_rot, goal_rot, object_angvel, object_pos, fingertip_pos):
    # Rotation reward
    rot_diff = torch.abs(torch.sum(object_rot * goal_rot, dim=1) - 1) / 2
    - rotation_reward_temp = 20.0
    + rotation_reward_temp = 30.0 Changing hyperparameter
    rotation_reward = torch.exp(-rotation_reward_temp * rot_diff)

    # Distance reward
    + min_distance_temp = 10.0
    min_distance = torch.min(torch.norm(fingertip_pos - object_pos[:, None], dim=2), dim=1).values
    - distance_reward = min_distance
    + uncapped_distance_reward = torch.exp(-min_distance_temp * min_distance)
    + distance_reward = torch.clamp(uncapped_distance_reward, 0.0, 1.0) Changing functional form

    - total_reward = rotation_reward + distance_reward
    + # Angular velocity penalty Adding new component
    + angvel_norm = torch.norm(object_angvel, dim=1)
    + angvel_threshold = 0.5
    + angvel_penalty_temp = 5.0
    + angular_velocity_penalty = torch.where(angvel_norm > angvel_threshold,
    +     torch.exp(-angvel_penalty_temp * (angvel_norm - angvel_threshold)), torch.zeros_like(angvel_norm))
    +
    + total_reward = 0.5 * rotation_reward + 0.3 * distance_reward - 0.2 * angular_velocity_penalty

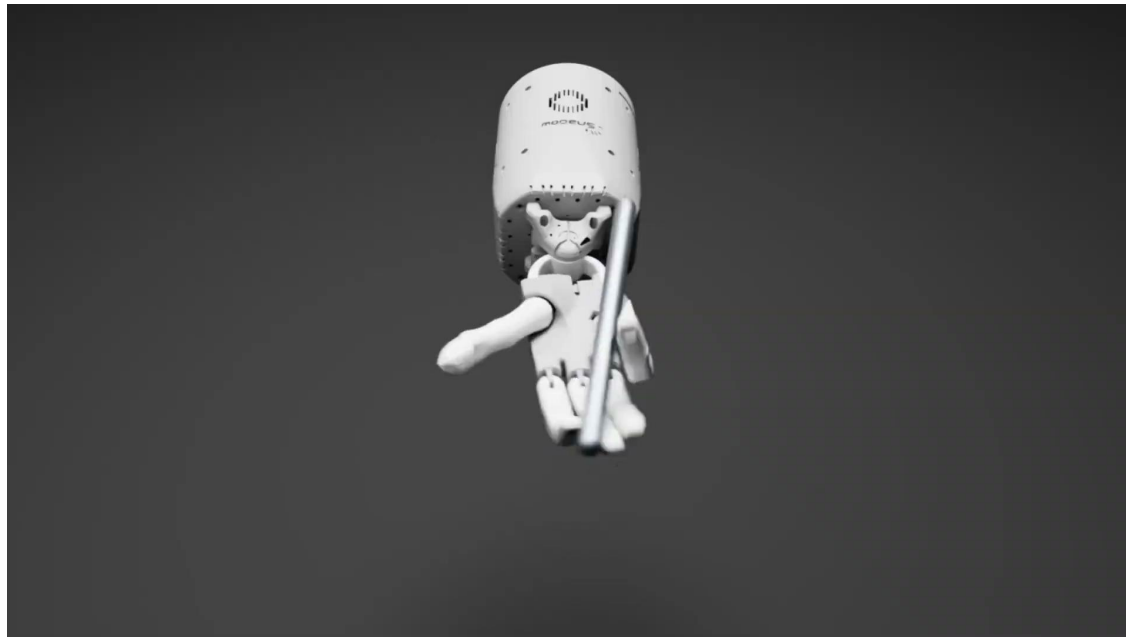
    reward_components = {
        "rotation_reward": rotation_reward,
        "distance_reward": distance_reward,
    +     "angular_velocity_penalty": angular_velocity_penalty,
    }

    return total_reward, reward_components
```

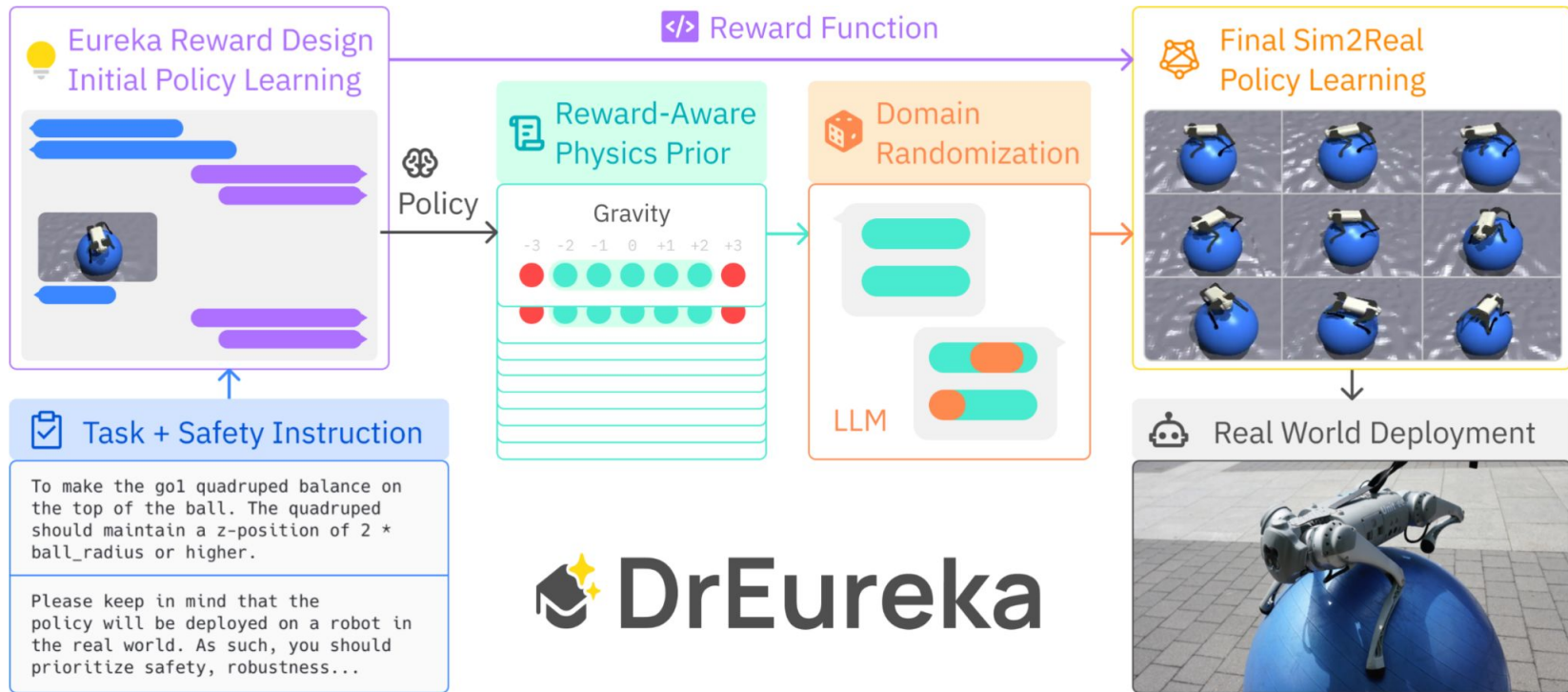
Eureka

Key results of the method:

- outperforms human reward on a wide range of environments
- consistently improves over time
- generates novel rewards compared to human rewards
- improves from human feedback



DrEureka

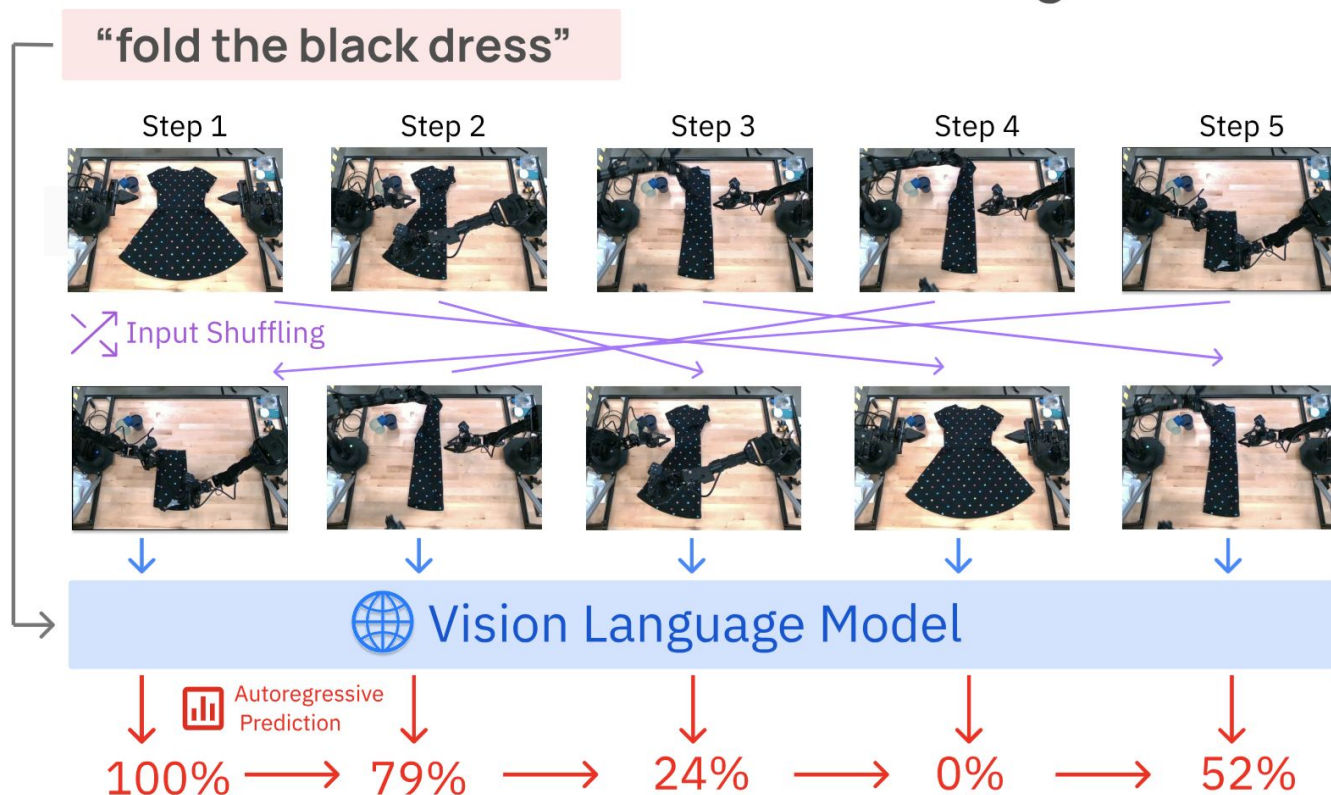


DrEureka

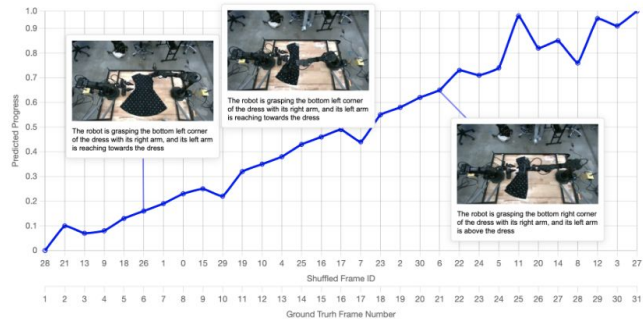


Generative Value Learners

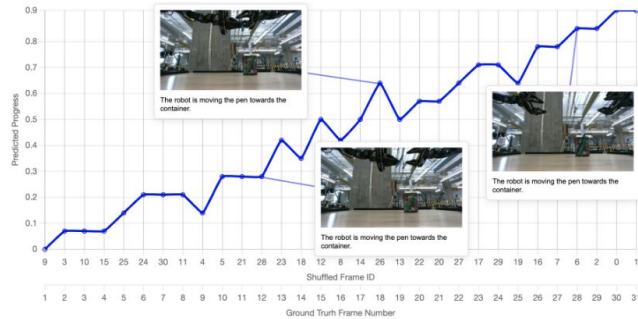
Generative Value Learning



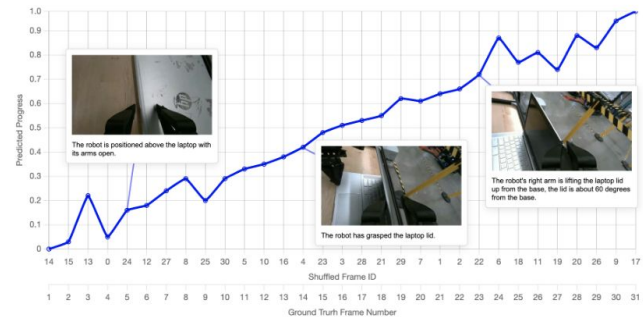
Generative Value Learners



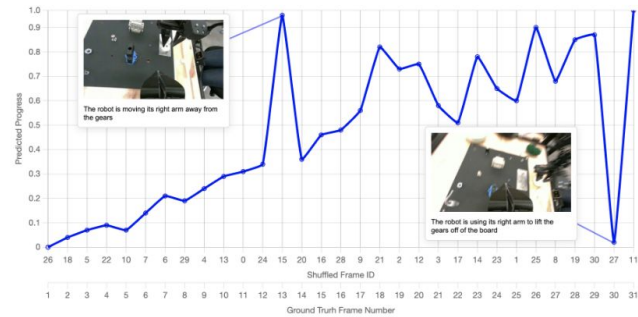
(a) `fold_dress` from the top-down view.



(b) `pen_handover` from the table view.



(c) `close_laptop` from the right wrist camera.



(d) `remove_gears` from the left wrist camera.

Figure 5 | Example GVL predictions on real-world ALOHA tasks. GVL can successfully unshuffle video frames and generate meaningful task values on diverse tasks and camera viewpoints.

(Some) Open questions

- Unified, effective, reproducible evaluation
- Sim-to-real gap
- Efficient collection and usage of human demonstration data
- High inference time of foundation models
- Long-horizon task planning
- Ensuring robustness and safety of deployed models



Conclusion

- Embodied AI is a research area at the intersection of NLP, CV and RL
- Embodied AI moves forward large-scale generalist models
- EAI models enable robot to perceive the world, talk, reason and act
- Evaluation of EAI models in general is a very challenging task
- Despite a decade of the rapid progress in NLP and CV, EAI systems (**understanding the world, planning and acting**) are in the beginning of their development

